



UNIVERSITY OF PISA

DEPARTMENT OF INFORMATION ENGINEERING

Master's Thesis in Artificial Intelligence and Data Engineering

Quality-based Index Tiering for Efficient Dense Retrieval

Supervisors:
Prof. Nicola Tonellotto
Dr. Francesca Pezzuti

Candidate:
Matteo Comini

ACADEMIC YEAR 2025/2026

Abstract

Modern search systems increasingly rely on dense retrieval, but collections keep expanding while the cost of storing and scanning high-dimensional representations (memory, latency, and energy) becomes harder to sustain. At the same time, corpora contain substantial amounts of redundant or weakly informative passages that often contribute little to retrieval quality, yet can still matter for a subset of queries. This creates a persistent effectiveness–efficiency trade-off: static pruning lowers cost but can harm coverage in the long tail, where less frequent information needs still require access to otherwise low-utility content.

This thesis studies a quality-aware alternative that sits between indexing the entire collection in a single dense index and static pruning. Each passage is assigned a scalar quality score (QualT5) and the corpus is partitioned into high- and low-quality pools via controlled, percentile-based splits. Retrieval is organized as a selective pipeline: a first pass searches the high-quality pool, while a query-level fallback selectively accesses the low-quality pool when needed. We compare two policies that instantiate this idea: (i) Two-Tier retrieval with two physical FAISS IVF indexes and explicit fallback, and (ii) Virtual Partitioning, which uses a single index and enforces partition membership through ID-based filtering and an optional second pass. Experiments on MSMARCO evaluate effectiveness (NDCG@10, MRR@10) alongside efficiency metrics under systematic sweeps of the high-tier budget and ANN search effort (nprobe), and analyze how QualT5 score distributions (tiny vs base) affect the resulting splits and the attainable trade-offs.

Overall, the thesis provides a reproducible assessment of when quality-based partitioning with selective fallback can reduce average retrieval cost while preserving robust effectiveness, and how system design choices influence this trade-off.

Contents

List of Acronyms	13
1 Introduction	19
1.1 General context and motivation	19
1.2 Thesis goals	20
1.3 Thesis structure	21
2 Background and Related Work	23
2.1 Information Retrieval and IR Systems	23
2.1.1 Overview of an IR System	24
2.1.2 Text processing and indexing basics	25
2.2 Lexical Retrieval	25
2.2.1 TF-IDF and BM25	25
2.2.2 Advantages and Limitations	26
2.2.3 Transition to Neural Retrieval	27
2.3 Transformer Architecture	27
2.3.1 Self-attention and Transformer blocks	27
2.3.2 Encoder-decoder models and T5	29
2.4 Neural Information Retrieval	30
2.4.1 Word Embeddings	31
2.4.2 Interaction-focused Systems	32
2.4.3 Representation-focused Systems	33
2.4.4 Evaluation of IR Systems	38
2.5 Quality Estimation for Retrieval: QualT5	40
2.5.1 From query-conditioned relevance to query-agnostic quality	41
2.5.2 QualT5 as a supervised passage quality estimator	41
2.5.3 Static pruning with a quality threshold	42
2.5.4 Model size and efficiency trade-offs	43
2.5.5 Positioning within retrieval pipelines	43

2.6	Similarity Search and Indexing with FAISS	43
2.6.1	Exact search (brute force)	44
2.6.2	Inverted files (IVF)	44
2.6.3	Graph-based methods (HNSW)	48
2.6.4	Compression and quantization (PQ/OPQ)	49
3	Research Objectives	52
3.1	Problem setting: quality-aware corpus partitioning for efficient retrieval	52
3.2	Research Questions and Experimental Analyses	53
3.2.1	RQ1: QualT5 quality signal—distribution, separability, and tiny vs base	53
3.2.2	RQ2: Architectural design—Two-Tier vs Virtual Partitioning	54
3.2.3	RQ3: Final performance—effectiveness vs efficiency across partition thresholds	54
4	Methodology	56
4.1	Method overview	56
4.2	Problem formulation	57
4.3	Index construction paradigms	57
4.3.1	Two-Tier index construction	58
4.3.2	Virtual Partitioning index construction	58
4.4	Retrieval architectures and baseline	59
4.4.1	Baseline	59
4.4.2	Two-Tier execution	59
4.4.3	Virtual Partitioning execution	59
4.5	Gating and query-time outcomes	60
5	Experimental Setup	63
5.1	Dataset	63
5.2	Quality score models	63
5.3	Dense representations	64
5.4	Tooling	64
5.5	Index creation	64
5.5.1	IVF instead of HNSW: scalability and build-time considerations	64
5.6	Search-time controls and experimental sweeps	65
5.7	Measurements	65

6	QualT5 quality signal:	
	distribution, separability, and tiny vs base (RQ1)	67
6.1	Global score distributions: skewness and limited separability at the high end	67
6.1.1	Tiny vs. Base: calibration shift and concentration	68
6.2	Cluster-conditioned behavior: heterogeneity across IVF coarse partitions	69
6.3	Practical impact of the induced partitions (tiny vs. base)	71
7	Architectural design:	
	Two-Tier vs Virtual Partitioning (RQ2)	74
7.1	Rationale and comparison dimensions	74
7.2	Two-Tier: two physical indexes and query-level fallback	75
7.3	Virtual Partitioning: one physical index with logical filtering and shared coarse assignment	75
7.4	Fallback (gating) mechanism: margin and entropy as uncertainty signals	76
7.5	Pros and cons of the two approaches (summary)	77
8	Final performance:	
	effectiveness vs efficiency across partition thresholds (RQ3)	80
8.1	Experimental setting and comparison protocol	80
8.2	Margin vs. Entropy as Gating Signals	81
8.2.1	TwoTier: effectiveness–latency trade-offs under different splits	81
8.2.2	Virtual Partitioning: robustness of the gating choice	82
8.3	Moderate-cost regime: nprobe =128	84
8.3.1	Two-Tier: effectiveness–latency curve	84
8.3.2	Virtual Partitioning: effectiveness–latency curve	84
8.3.3	Practical reading of the frontier (nprobe =128)	85
8.4	High-cost regime: nprobe =8192	86
8.4.1	Two-Tier: approaching baseline quality under high search effort	86
8.4.2	Virtual Partitioning: comparison under high search effort	87
8.4.3	Practical reading of the frontier (nprobe =8192)	87
8.5	Comparative synthesis and recommended operating points	88
8.6	Frontier visualization: NDCG@10 and MRR@10 vs latency	88
8.7	Answer to RQ3	93

9	Conclusions and Future works	95
9.1	Summary of findings	95
9.2	General conclusions	96
9.3	Future works	96

List of Figures

2.1	The Transformer encoder–decoder architecture. Adapted from Vaswani et al. [1].	28
2.2	Scheme of an interaction-focused model based on convolutional neural networks [37].	33
2.3	Batch construction strategies for dense retriever training: (a) random sampling often yields mostly easy in-batch negatives; (b) Topic Aware Sampling (TAS) increases topical competition by sampling queries from the same cluster; (c) TAS-Balanced further improves supervision by sampling training pairs across a balanced range of teacher-score margins [45].	36
2.4	Overview of ColBERT: independent encoding of query and document into contextual token embeddings and late interaction scoring [41].	37
2.5	QualT5- quality estimation of the passages [54].	41
2.6	Illustration of inverted-file (IVF) indexing. The dataset is partitioned into clusters during indexing, and at query time the search is restricted to a subset of nearby clusters determined by the query [60].	45
2.7	Left: illustration of an HNSW graph with three connection lengths—short links in grey, longer links in green, and the longest links in red. E denotes the entry point. Right: the same HNSW represented as a stack of three layers; dotted lines indicate the corresponding position of each vector in the layer below [62].	48
2.8	IVF-PQ indexing pipeline: vectors are assigned to IVF inverted lists, residuals w.r.t. the list centroid are computed and PQ-encoded, and the resulting codes are stored in the corresponding inverted lists [65].	50

4.1	Two-Tier index creation: passages are scored offline, ordered by quality, then partitioned into a high tier and a low tier, resulting in two physical indexes.	58
4.2	Virtual Partitioning index creation: passages are scored offline, indexed in a single full index, and quality scores define virtual high/low partitions (enforced at query time via filtering).	58
4.3	Closed-gate outcome: the gating mechanism does not activate fall-back, so retrieval terminates after the high-tier pass and returns high-tier top- k results only.	60
4.4	Open-gate outcomes: when uncertainty is detected, the system triggers a low-tier pass and merges candidates to produce the final top- k	61
6.1	Global distribution of QualT5 scores for tiny and base. Both distributions are strongly left-skewed (mass near the upper bound with a tail toward low scores); base is more concentrated near the upper end.	68
6.2	Distribution of cluster-specific thresholds required to obtain a fixed low-tier share within each IVF list of the complete index. The dashed line denotes the corresponding global threshold, highlighting cross-list heterogeneity.	70
8.1	TwoTier trade-offs (NDCG@10 vs. average time) comparing margin- and entropy-based gating across different splits. Points are annotated with activation rates; the Full Index baseline is shown as reference.	82
8.2	Virtual Partitioning trade-offs (NDCG@10 vs. average time) comparing margin- and entropy-based gating across different splits. As in TwoTier, the dominant effect is driven by activation rather than by the specific signal choice.	83
8.3	Two-Tier: NDCG@10 vs retrieval latency (nprobe =128).	89
8.4	Two-Tier: NDCG@10 vs retrieval latency (nprobe =8192).	89
8.5	Virtual Partitioning: NDCG@10 vs retrieval latency (nprobe =128).	90
8.6	Virtual Partitioning: NDCG@10 vs retrieval latency (nprobe =8192).	90
8.7	Two-Tier: MRR@10 vs retrieval latency (nprobe =128).	91
8.8	Two-Tier: MRR@10 vs retrieval latency (nprobe =8192).	91
8.9	Virtual Partitioning: MRR@10 vs retrieval latency (nprobe =128).	92
8.10	Virtual Partitioning: MRR@10 vs retrieval latency (nprobe =8192).	92

List of Tables

6.1	HighOnly Virtual Partitioning induced by QualT5-base (nprobe=128).	72
6.2	HighOnly Virtual Partitioning induced by QualT5-tiny (nprobe=128).	72
8.1	Two-Tier with nprobe =128: effectiveness vs latency across p and margin-gating thresholds.	84
8.2	Virtual Partitioning with nprobe =128: effectiveness vs latency across p and margin-gating thresholds.	85
8.3	Two-Tier with nprobe =8192: effectiveness vs latency across p and margin-gating thresholds.	86
8.4	Virtual Partitioning with nprobe =8192: effectiveness vs latency across p and margin-gating thresholds.	87

List of Acronyms

ADC	Asymmetric Distance Computation
ANCE	Approximate Nearest neighbor Negative Contrastive Learning
ANN	Approximate Nearest Neighbor
AP	Average Precision
BERT	Bidirectional Encoder Representations from Transformers
BERTDOT	BERT-based dual-encoder with dot-product scoring
BM25	Best Matching 25
CIKM	Conference on Information and Knowledge Management
COIL	COntextualized Inverted List
ColBERT	Contextualized Late Interaction over BERT
DCG	Discounted Cumulative Gain
DPR	Dense Passage Retrieval
EMNLP	Empirical Methods in Natural Language Processing
ERR	Expected Reciprocal Rank
FAISS	Facebook AI Similarity Search
FFN	Feed-Forward Network
GPU	Graphics Processing Unit
GRU	Gated Recurrent Unit
HNSW	Hierarchical Navigable Small World
ID	Identifier
IDCG	Ideal Discounted Cumulative Gain
IDF	Inverse Document Frequency
IR	Information Retrieval
IVF	Inverted File Index
k-NN	k-Nearest Neighbor
LSTM	Long Short-Term Memory
MAP	Mean Average Precision
MRR	Mean Reciprocal Rank

MS MARCO Microsoft MACHine Reading Comprehension
NDCG Normalized Discounted Cumulative Gain
NLP Natural Language Processing
OPQ Optimized Product Quantization
PQ Product Quantization
QA Question Answering
QPS Queries Per Second
qrels Query relevance labels (judgments)
QualT5 Quality-aware T5 model
RBP Rank-Biased Precision
RNN Recurrent Neural Network
RQ Research Question
RR Reciprocal Rank
STAR Spatio-Temporal Alignment and Representation
T5 Text-to-Text Transfer Transformer
TAS Topic Aware Sampling
TAS-B Topic Aware Sampling - Balanced
TF-IDF Term Frequency-Inverse Document Frequency
TREC Text RETrieval Conference
TT Two-Tier
VP Virtual Partitioning
YAML Yet Another Markup Language

List of Symbols

- q A user query representing a specific information need.
- d, p A document or a passage within the corpus.
- D, P The document collection or the full corpus of passages.
- $s(q, d)$ The retrieval score computed for document d given query q .
- N Total number of documents in the collection.
- n_t Number of documents in the collection containing term t .
- $|d|$ Length of document d expressed in tokens or terms.
- $f_{t,d}$ Number of occurrences (frequency) of term t in document d .
- tf, idf Term frequency and inverse document frequency components.
- $avgdl$ Average document length across the entire corpus.
- k_1, b Hyperparameters for the BM25 retrieval model.
- Q, K, V Query, Key, and Value matrices in the self-attention mechanism.
- d_k Dimensionality of the keys used for scaling dot-product attention.
- W^Q, W^K, W^V, W^O Learned projection matrices for the multi-head attention mechanism.
- e_t Continuous dense vector representation (embedding) of term t .
- $\langle u, v \rangle$ Inner product (or dot product) between two vectors u and v .
- $\|u\|$ Euclidean (L_2) norm of vector u .
- $\cos(u, v)$ Cosine similarity between vectors u and v .
- θ Angle between vectors or model parameters to be learned.
- $\phi(\cdot), \psi(\cdot)$ Encoder functions for mapping queries and documents to a dense vector space.
- E_q, E_d Sets of contextual token embeddings for multiple-representation models.
- rel_i Binary relevance indicator for the document at rank i .
- g_i Graded relevance label assigned to the i -th ranked document.
- R Total number of relevant documents for a given query.
- $l(q)$ End-to-end execution latency for processing query q .
- T_{build} Total wall-clock time required for index construction.
- M_{total} Total memory footprint of the retrieval system.

$Qual(p)$ Global, query-agnostic quality score for passage p .
 $\mathcal{L}$ Loss function used during model training.
 $\mathcal{X}$ Database of vectors in the embedding space.
 $\mathcal{M}$ Set of centroids representing coarse regions of the space.
 μ_j Vector centroid of the j -th inverted list in an IVF structure.
 $q(x)$ Coarse quantization function mapping a vector to its nearest centroid.
 $\mathcal{P}(q)$ Set of promising inverted lists selected for probing.
 K, P Number of coarse clusters ($nlist$) and probed lists ($nprobe$).
 m Number of sub-vectors in Product Quantization or margin threshold.
 r Residual vector between an embedding and its list centroid.
 H_p, L_p High-quality and low-quality partitions induced by percentile p .
 p Budget parameter defining the retained share of the corpus.
 Δ Score margin between the top-1 and top-2 ranked candidates.
 H Shannon entropy of the normalized score distribution.
 α Threshold parameter for entropy-based fallback gating.
 τ Temperature parameter for sharpening the softmax distribution.
 p_i Softmax-normalized probability assigned to the i -th candidate.
 $\mathcal{T}$ Grid of pre-defined decision thresholds for gating evaluations.

Chapter 1

Introduction

1.1 General context and motivation

Information Retrieval (IR) systems sit at the core of how people and applications access knowledge in large text collections. From web search and question answering to enterprise and domain-specific search, the common requirement is to return relevant information quickly and reliably, even when the underlying corpus contains millions of heterogeneous and noisy passages. As neural methods have become increasingly dominant—thanks to their ability to capture semantic similarity beyond exact term overlap—first-stage retrieval is often framed as a vector search problem, where queries and passages are embedded in a shared space and ranked by similarity.

However, this progress comes with an immediate operational tension. Dense retrieval typically improves effectiveness, but it also increases computational and memory costs: large-scale embedding collections are expensive to store, and approximate nearest neighbor search still requires non-trivial query-time work to maintain good recall. As collections grow, the question is no longer only “how accurate is retrieval?”, but also “how can we sustain that accuracy under realistic efficiency constraints?”. In this setting, naïvely searching the entire index for every query becomes increasingly costly, while aggressive pruning strategies—although attractive for reducing footprint and latency—risk removing passages that are rare but crucial for relevance.

This thesis is motivated by the idea that **not all passages contribute equally** to first-stage retrieval effectiveness, and that corpus-level heterogeneity can be exploited rather than ignored. Instead of adopting static pruning, the work explores a more controlled middle ground: **quality-aware corpus partitioning**, where passages are split into a “high-quality” tier and a “low-quality” tier

based on an automatic quality signal. The high tier is intended to concentrate more useful content and to be cheaper to search; the low tier is retained as a safety net to preserve coverage. The central intuition is that many queries can be answered effectively by searching a smaller, higher-quality subset, while only a minority of “hard” or ambiguous queries should trigger additional work on the low tier. This yields a practical research problem: **how to design tiered retrieval so that it reduces average cost without introducing brittle failures or unacceptable losses in ranking quality.**

1.2 Thesis goals

The goal of this thesis is to investigate whether, and under which conditions, *quality-aware* corpus organization can improve the effectiveness–efficiency trade-off of neural first-stage retrieval on large passage collections. To address this goal, the work is structured around three research questions, and it pursues three corresponding objectives.

1. **RQ1 – Quality signal perspective (“how to define a partition”).** The first objective is to assess the role of the passage quality signal used to induce the high/low split. The thesis analyzes how quality scores distribute across the corpus, how separable the resulting subsets are, and how different partitioning regimes may affect both coverage and the potential gains in efficiency.
2. **RQ2 – System perspective (“how to use a partition”).** The second objective is to study how different retrieval designs can exploit the same high/low quality partition to reduce average retrieval cost while preserving ranking quality. In particular, the thesis compares alternative ways of organizing search across tiers (e.g., separate tiers versus logical tiering) and evaluates their practical implications in terms of robustness and efficiency.
3. **RQ3 – End-to-end perspective (“when tiering works”).** The third objective is to provide an end-to-end evaluation of quality-aware tiering under realistic retrieval constraints. The thesis quantifies the resulting effectiveness–efficiency frontiers across multiple operating points, identifying the conditions under which tiered retrieval is beneficial relative to a full-index baseline and highlighting the main trade-offs and limitations.

1.3 Thesis structure

This thesis is organized into nine chapters.

Chapter 2 Chapter 2 provides the background needed to contextualize dense first-stage retrieval and the indexing mechanisms used in modern ANN search.

Chapter 3 Chapter 3 formalizes the problem setting and introduces the research objectives, defining the three research questions that guide the empirical analysis (RQ1–RQ3).

Chapter 4 Chapter 4 presents the methodology at the level of pipeline logic and execution semantics, describing how passage-quality scores induce high/low partitions and how tiered retrieval is instantiated through Two-Tier and Virtual Partitioning, including the gating mechanism that conditionally activates fallback.

Chapter 5 Chapter 5 reports the experimental setup in detail, including dataset, quality-score models, dense representations, indexing hyperparameters, sweep grids, and the evaluation and measurement protocol used throughout the thesis.

Chapter 6–8 Chapters 6–8 report results and answer the research questions: Chapter 6 analyzes the QualT5 quality signal—its distributional properties, separability, and the differences between tiny and base variants (RQ1); Chapter 7 focuses on the architectural comparison between Two-Tier and Virtual Partitioning (RQ2); Chapter 8 consolidates end-to-end findings by mapping the effectiveness–efficiency trade-off across partition thresholds and operating regimes (RQ3).

Chapter 9 Finally, Chapter 9 summarizes the main findings, discusses their implications and limitations, and outlines directions for future work.

Chapter 2

Background and Related Work

2.1 Information Retrieval and IR Systems

Information Retrieval (IR) studies how to search for useful information within large collections of mostly unstructured data, such as web pages, scientific articles, product descriptions, or short passages [2, 3]. In practice, IR methods power search engines and many downstream applications where users need to quickly access the most useful content for a task, including question answering, digital libraries, enterprise search, and domain-specific settings such as legal or biomedical retrieval.

An IR system can be described as a function that receives a user request and returns a ranked list of results. The user request is typically expressed as a *query*, which is only an imperfect proxy of the underlying *information need*. The system therefore aims to estimate which items in the collection best satisfy that need, and to order them accordingly. Formally, given a query q and a collection of documents D , the system computes a score $s(q, d)$ for each document $d \in D$ and outputs a ranking where higher-scoring documents appear earlier [2, 3].

A key concept in IR is *relevance*, i.e., the degree to which a document helps the user accomplish the goal behind the query. Relevance is inherently subjective and context-dependent: the same document may be relevant for one user and irrelevant for another, depending on intent, background knowledge, and constraints (time, novelty, authority, etc.). In experimental benchmarks, relevance is often approximated through human labels (binary or graded), which provide a practical but necessarily simplified view of real user satisfaction [4, 5].

Throughout this thesis, the following basic terminology is adopted:

- **Query.** A textual representation of the user request, expressed as keywords or natural language, intended to capture the information need [6, 7].

- **Document.** A retrievable unit in the collection (e.g., a full web page, a scientific abstract, or a passage) that may contain information useful for the query.
- **Corpus / Collection.** The full set of documents over which the retrieval system operates.
- **Relevance.** The degree of usefulness of a document with respect to the query and the underlying information need; it can be modeled as binary (relevant / not relevant) or graded.

From a system perspective, IR is usually framed as a trade-off between *effectiveness* and *efficiency* [2, 3]. Effectiveness concerns whether the system retrieves and ranks the “right” documents (according to relevance judgments or user satisfaction), while efficiency concerns whether it can do so within realistic resource constraints, such as latency, memory, and throughput. These two dimensions become especially critical at scale, where a naive exhaustive scoring of all documents is typically infeasible, motivating efficient indexing and ranking strategies under practical constraints [2, 3].

2.1.1 Overview of an IR System

A classical IR system can be viewed as a pipeline that, given a user query q , searches a large document collection and returns a ranked list of results. The overall objective is to identify documents that best match the query while keeping query-time processing fast enough for practical use.

Offline stage: indexing the collection. Before any query is issued, documents are processed and represented in a form suitable for search. The system then builds an *index* that makes it possible to efficiently access only a small portion of the collection at query time, rather than scanning all documents [2, 3].

Online stage: query processing and retrieval. When a query arrives, it is analyzed in a way consistent with the document processing (e.g., tokenization and normalization). The system then uses the index to retrieve candidate documents that share evidence with the query terms and assigns them a retrieval score. The final output is obtained by sorting documents by this score and returning the top-ranked results [2, 3].

Overall, classical IR systems rely on efficient indexing and scoring functions to approximate relevance and to provide ranked results over large collections within strict time constraints.

2.1.2 Text processing and indexing basics

Classical IR systems rely on a lightweight text processing pipeline to make documents and queries comparable. Common steps include *tokenization* (splitting text into terms), *normalization* (e.g., lowercasing and handling punctuation), and optional removal of high-frequency terms (*stopwords*). In some settings, *stemming* or *lemmatization* is applied to reduce morphological variants to a common form, with the aim of improving matching between queries and documents [2, 8].

After preprocessing, the collection is typically organized through an *inverted index* [2, 3]. For each term t , the index stores a posting list containing the identifiers of documents in which t appears, often enriched with statistics such as term frequency (and, in some implementations, term positions). This structure enables efficient retrieval by allowing the system to access only the documents associated with the query terms. At query time, the posting lists are traversed and combined to compute document scores, and the top-ranked documents are returned.

2.2 Lexical Retrieval

2.2.1 TF-IDF and BM25

Lexical retrieval models estimate relevance by exploiting *exact* term overlap between a query and a document. Under a bag-of-words assumption, documents and queries are represented as sparse term statistics, and scoring is computed efficiently using an inverted index [2].

TF-IDF. TF-IDF assigns a weight to each term by combining a *term frequency* component (how prominent the term is in a document) with an *inverse document frequency* component (how informative the term is across the corpus) [2, 9]. A common formulation is:

$$\text{tfidf}(t, d, D) = \text{tf}(t, d) \cdot \text{idf}(t, D), \quad (2.1)$$

where a length-normalized term frequency can be defined as:

$$\text{tf}(t, d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}}, \quad (2.2)$$

and the inverse document frequency as:

$$\text{idf}(t, D) = \log \left(\frac{N}{|\{d \in D : t \in d\}|} \right). \quad (2.3)$$

Here, t is a term, d a document, D the document collection, $f_{t,d}$ the number of occurrences of t in d , and $N = |D|$ the total number of documents.

BM25. BM25 can be seen as a stronger lexical baseline that refines TF-IDF by introducing (i) *term-frequency saturation* (additional occurrences of a term yield diminishing returns) and (ii) *document length normalization*. Its score for a query q and document d is:

$$\text{BM25}(q, d) = \sum_{t \in q} \text{idf}(t) \cdot \frac{f_{t,d} \cdot (k_1 + 1)}{f_{t,d} + k_1 \cdot \left(1 - b + b \cdot \frac{|d|}{\text{avgdl}}\right)}, \quad (2.4)$$

with an IDF variant commonly defined as:

$$\text{idf}(t) = \log \left(\frac{N - n_t + 0.5}{n_t + 0.5} + 1 \right). \quad (2.5)$$

In this notation, $|d|$ is the length of document d (in tokens/terms), avgdl is the average document length in the corpus, n_t is the number of documents containing term t , and k_1 and b are hyperparameters (often $k_1 \in [1.2, 2.0]$ and $b \approx 0.75$).

In practice, both TF-IDF and BM25 are attractive because they are simple, efficient at scale, and their scoring behavior is interpretable in terms of explicit term statistics [10, 11].

2.2.2 Advantages and Limitations

Lexical retrieval remains a strong baseline because it is efficient, simple to deploy, and effective when queries contain discriminative keywords. In addition, lexical scores are relatively transparent, since they are driven by explicit term statistics (e.g., term frequency and document frequency), which makes the behavior of the system easier to interpret and tune.

At the same time, lexical matching has inherent limitations. Because relevance is estimated primarily through surface term overlap, lexical models struggle with *semantic generalization*[12, 13]: relevant documents may use synonyms, paraphrases, or different levels of specificity and therefore receive low scores. Conversely, exact matches can be misleading when terms are ambiguous (*polysemy*) or

appear in a document without addressing the underlying information need. These effects are especially visible in settings with short or underspecified queries, where relying on exact overlap may miss conceptually relevant evidence [13].

2.2.3 Transition to Neural Retrieval

The limitations of term matching motivate a transition toward neural IR approaches, which aim to move beyond lexical overlap. In neural retrieval, queries and documents are embedded into dense vector spaces using deep language models, enabling matches based on conceptual similarity. This capability is increasingly central in modern IR pipelines, particularly for demanding tasks such as question answering and open-domain search, where relevant content often differs substantially from the query in surface form [14, 15].

2.3 Transformer Architecture

The Transformer is a sequence-to-sequence architecture that replaces recurrence and convolutions with attention mechanisms (Figure 2.1), enabling highly parallel computation and effective modeling of long-range dependencies [1]. Compared to earlier NLP sequence-to-sequence models based on recurrent networks (e.g., LSTMs/GRUs) or convolutional stacks, this design removes the inherently sequential computation across time steps, which had limited parallelism and made training less efficient on long sequence [16–18]. In its original form, it follows an encoder–decoder design in which the encoder produces contextual representations of the input sequence, and the decoder generates the output autoregressively while attending to the encoder states [1].

2.3.1 Self-attention and Transformer blocks

Attention and self-attention. An attention function maps a *query* and a set of *key–value* pairs to an output vector. The output is a weighted sum of the values, where weights are computed by a compatibility measure between the query and each key. Self-attention is the special case in which queries, keys, and values are all derived from the same sequence, allowing each position to incorporate information from other positions in that sequence [19, 20].

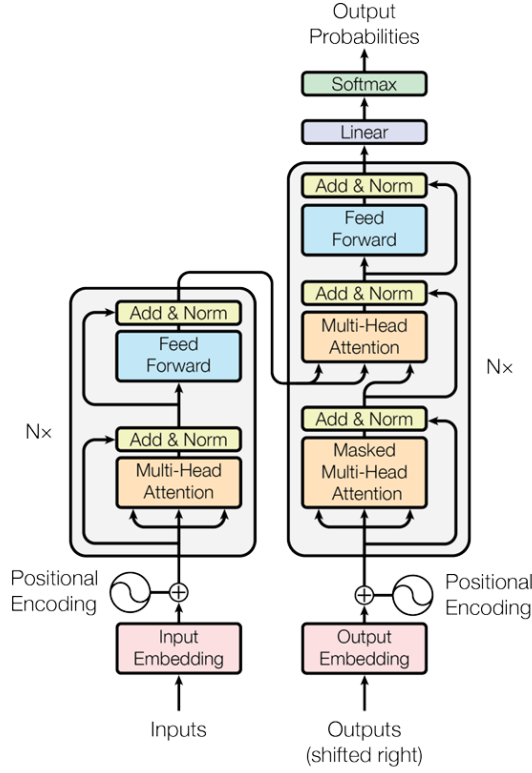


Figure 2.1: The Transformer encoder–decoder architecture. Adapted from Vaswani et al. [1].

Scaled dot-product attention. Given matrices of queries Q , keys K , and values V , the Transformer computes attention as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V, \quad (2.6)$$

where d_k is the key dimensionality used for scaling [1].

Multi-head attention. Instead of a single attention operation, the model uses multiple heads operating in parallel on different learned projections:

$$\begin{aligned} \text{MultiHead}(Q, K, V) &= \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O, \\ \text{head}_i &= \text{Attention}(QW_i^Q, KW_i^K, VW_i^V). \end{aligned} \quad (2.7)$$

This improves expressiveness by allowing attention over different representation subspaces [1].

Core Transformer block. Both encoder and decoder stacks are built from repeated blocks that combine attention with a position-wise feed-forward network

(FFN) [1]. The FFN is applied independently to each position:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2. \quad (2.8)$$

To preserve order information, positional encodings are added to the input embeddings [1].

Encoder–decoder blocks. The encoder–decoder configuration can be interpreted as a design that separates *understanding* from *generation*. The encoder processes the entire input with bidirectional self-attention, producing a compact set of contextual states. The decoder then generates the output left-to-right: masked self-attention ensures that predictions at time t depend only on previously generated tokens, while cross-attention allows each decoding step to selectively focus on the most relevant parts of the encoded input. The remaining feed-forward sub-layer provides additional non-linearity at each position. Together, these components form a block that is repeated across layers to gradually refine the conditional generation process [1].

2.3.2 Encoder–decoder models and T5

Architectural configurations. Transformer-based models can be instantiated in different architectural configurations, depending on how self-attention is masked and on whether the model includes an explicit *encoder–decoder* separation. A useful high-level taxonomy includes three common variants [1].

- **Encoder-only.** A stack of Transformer encoder layers with *fully-visible* self-attention, where each token can attend to all other tokens in the input sequence. The model produces contextual representations that are particularly suited for understanding-oriented tasks (e.g., classification or scoring) where the entire input can be processed bidirectionally [21].
- **Decoder-only.** A single stack of Transformer decoder layers with *causal* self-attention. The attention mask prevents each position from attending to future tokens, enabling left-to-right generation and next-token prediction. This configuration is typically used for language modeling and text generation [22].
- **Encoder–decoder.** Two stacks are used: an encoder with fully-visible attention that encodes the input sequence, and a decoder with causal self-attention that generates the output sequence token-by-token. In addition, each decoder layer includes *cross-attention* over the encoder outputs, al-

lowing the decoder to condition generation on the entire encoded input. This setup naturally models sequence-to-sequence problems (e.g., translation, summarization, question answering) where an input text is transformed into an output text [1, 23].

T5 as an encoder–decoder instantiation. T5 is a representative encoder–decoder Transformer that adopts a unified *text-to-text* interface: both inputs and targets are expressed as text sequences, and task differences are handled through lightweight textual prefixes [23]. Architecturally, this choice avoids task-specific prediction heads and keeps the same encoder–decoder backbone across applications. Training is performed with a denoising objective compatible with sequence-to-sequence generation: the encoder receives a corrupted input (with spans replaced by sentinel tokens), and the decoder reconstructs the missing content [23]. As a result, T5 pretraining aligns with the encoder–decoder computation pattern (encoding + conditional decoding), providing a general-purpose model that can be adapted without changing the architecture.

2.4 Neural Information Retrieval

Classical IR systems based on lexical matching (e.g., TF–IDF and BM25) are strong and efficient, but they are fundamentally constrained by *vocabulary mismatch*: a document can be relevant even when it does not contain the exact query terms (e.g., due to synonyms, paraphrases, or different levels of specificity) [24, 25]. Neural Information Retrieval (Neural IR) addresses this limitation by using deep neural networks to model relevance in a way that captures semantic relationships beyond surface term overlap. In practice, neural models can learn richer representations of queries and documents from data, reducing the reliance on hand-crafted matching heuristics and enabling more robust retrieval in settings where lexical overlap is sparse or misleading.

A useful way to view Neural IR is through what is being learned. On the one hand, neural models can learn *ranking functions* that directly estimate relevance between a query and a document by exploiting contextual interactions. On the other hand, they can learn *representations* of queries and documents (typically embeddings) such that semantic similarity can be computed efficiently in a shared vector space. These two perspectives lead to different system designs with different trade-offs: interaction-heavy models tend to be more accurate but computationally expensive, while representation-based approaches are designed to

scale by pre-computing document representations and using efficient vector search [25, 26].

Modern search engines therefore often adopt *multi-stage pipelines*: an efficient first stage generates a candidate set (using lexical retrieval and/or neural dense retrieval), and a more expensive neural model refines the ranking of the top candidates. This architecture allows Neural IR methods to improve retrieval effectiveness while still meeting practical constraints on latency, memory, and throughput. The following subsections introduce the main building blocks behind Neural IR, starting from word embeddings, and then covering interaction-focused and representation-focused systems [27, 28].

2.4.1 Word Embeddings

Early IR representations such as bag-of-words encode text as sparse vectors over the vocabulary. While effective and efficient, these sparse encodings provide no graded notion of relatedness: terms are treated as independent dimensions, making it hard to compare texts beyond exact overlap. Word embeddings are motivated by the *distributional hypothesis*, according to which words that occur in similar contexts tend to have similar meanings. By leveraging large text collections, embeddings learn continuous representations that capture such regularities directly from data [29, 30].

Definition and intuition. A word embedding is a mapping from a vocabulary term t to a dense vector $\mathbf{e}_t \in \mathbb{R}^n$, where n is typically a few hundred dimensions. Unlike sparse one-hot encodings, these vectors place words in a continuous space where semantic relatedness is reflected by geometric proximity: words with similar usage patterns tend to have vectors pointing in similar directions or with large inner products [31, 32].

Similarity in embedding spaces. Given two embeddings $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$, a fundamental similarity measure is the *inner product* (in Euclidean spaces, equivalent to the dot product):

$$\langle \mathbf{u}, \mathbf{v} \rangle = \mathbf{u}^\top \mathbf{v} = \sum_{i=1}^n u_i v_i. \quad (2.9)$$

Geometrically, $\langle \mathbf{u}, \mathbf{v} \rangle = \|\mathbf{u}\| \|\mathbf{v}\| \cos \theta$, where θ is the angle between the vectors. Hence, the inner product grows when vectors are aligned (small θ), but also when their norms are large, meaning that both direction and magnitude influence the score [33].

To focus purely on directional similarity, one often uses *cosine similarity*:

$$\cos(\mathbf{u}, \mathbf{v}) = \frac{\langle \mathbf{u}, \mathbf{v} \rangle}{\|\mathbf{u}\| \|\mathbf{v}\|} = \frac{\sum_{i=1}^n u_i v_i}{\sqrt{\sum_{i=1}^n u_i^2} \sqrt{\sum_{i=1}^n v_i^2}}. \quad (2.10)$$

Cosine similarity lies in $[-1, 1]$ and measures how closely two vectors point in the same direction. In practice, this makes it convenient for comparing representations when vector length should not dominate the score [34, 35].

Using embeddings in Neural IR. In Neural IR, embeddings provide a first step toward continuous semantic matching by turning text into continuous representations that can be compared directly. A simple approach is to obtain a single vector for a query and a document by aggregating the embeddings of their words, and then use the resulting vector similarity as a proxy for relevance. Even though this strategy is relatively coarse (it ignores word order and the fact that the meaning of a term can change with context), it captures an important intuition: two texts can be considered related even when they do not share the same surface vocabulary. For this reason, word embeddings are often introduced as a conceptual bridge from purely lexical IR toward neural methods that learn richer representations and matching functions from data [31, 36].

2.4.2 Interaction-focused Systems

Interaction-focused Neural IR models estimate relevance by explicitly modeling how a query and a document match at a fine granularity. Instead of collapsing each text into a single vector early on, these approaches build a joint query–document representation and let the network learn which token-level correspondences correlate with relevance [24, 37]. A common abstraction is an *interaction matrix*, whose entries represent similarities between query and document terms (e.g., computed from embeddings). The model then transforms this matrix into a single relevance score by learning how to combine local and global matching evidence.

Different families of interaction-focused methods mainly differ in how they summarize or smooth the raw similarity signal. *Histogram-based* approaches[38] aggregate, for each query term, the distribution of similarities observed across document terms and feed these binned statistics into a small neural model. *Kernel-based* variants[39] replace hard bins with differentiable kernels (e.g., Gaussian kernels), producing a soft notion of match strength across similarity ranges and allowing end-to-end learning. Other methods treat the interaction matrix as a structured 2D object and apply *convolutional* layers and pooling to capture proximity pat-

terns and higher-order matching cues (analogous to n -gram alignments), before mapping the resulting features to a score [40].

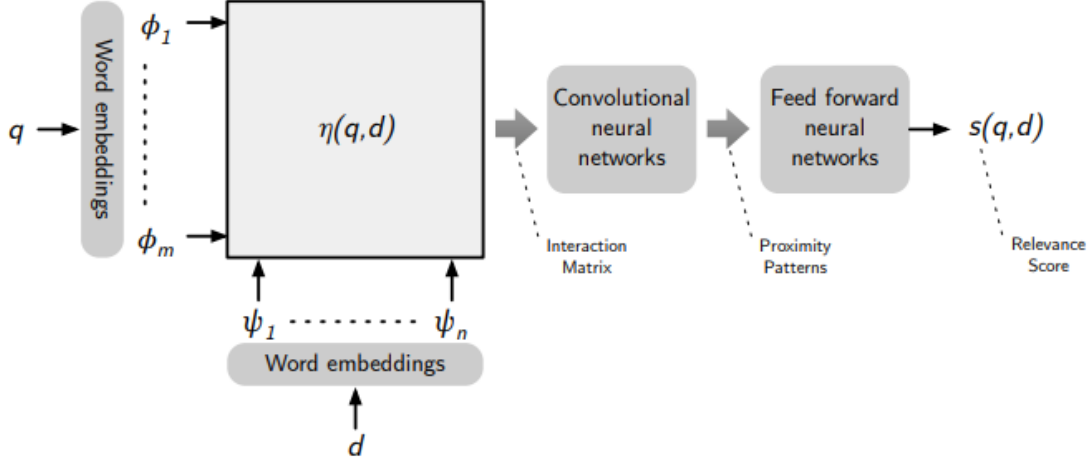


Figure 2.2: Scheme of an interaction-focused model based on convolutional neural networks [37].

Because they require joint query–document processing at inference time, interaction-focused models are typically used as re-rankers over a limited candidate set [27]. This motivates representation-focused retrieval, which shifts most computations offline.

2.4.3 Representation-focused Systems

The interaction-focused paradigm discussed above can capture rich matching patterns, but it requires joint processing of the query and each candidate document. This makes it computationally expensive and difficult to scale as a first-stage retriever over very large corpora. Representation-focused systems address this limitation by learning *separate* representations for queries and documents, so that most of the computation can be moved offline: document vectors are pre-computed and indexed, while at query time the system only encodes the query and performs efficient nearest-neighbor search in a shared embedding space.

Within representation-focused retrieval, a central design choice concerns how a text is represented in the embedding space. In *single-representation* (single-vector) models, each query and each document is mapped to a single dense vector, enabling highly efficient approximate nearest-neighbor retrieval [15]. In *multiple-representation* (multi-vector) models, a document (and sometimes the query) is represented by multiple vectors and relevance is computed through a late-interaction aggregation, trading higher computational cost for stronger ex-

pressiveness [41]. The next subsections detail these two approaches and discuss representative model families for each setting.

2.4.3.1 Single-representation dense retrieval

A first and widely adopted family of representation-focused systems is based on *single-representation* encoders, often referred to as *bi-encoders* or *dual-encoders*. The key idea is to map the query and each document (or passage) into a *single* dense vector, so that relevance can be computed with a simple similarity function. Concretely, the model learns a query encoder $\phi(\cdot)$ and a document encoder $\psi(\cdot)$, and scores a document d for a query q as:

$$s(q, d) = \phi(q)^\top \psi(d). \quad (2.11)$$

This makes retrieval efficient in practice, as scoring reduces to a nearest-neighbor search in the embedding space [37].

Common training pattern. Single-representation retrievers are usually trained with contrastive objectives where each query is paired with at least one relevant passage and many non-relevant passages. Negatives can be sampled randomly, mined as *hard negatives* using a retrieval system, or obtained through *in-batch* negatives (where positives of other queries in the batch act as negatives). This general recipe underlies several well-known dense retrievers such as DPR, ANCE, STAR, RocketQA, and related variants [15, 42–44]. In this thesis, the focus will be on TAS-B, which is a strong and practical instantiation of this family.

TAS-B: efficiently teaching a BERTDOT dense retriever. TAS-B (often presented as *TAS-Balanced*) is designed to train a highly effective single-representation dense retriever with *modest hardware*, while preserving the low-latency benefits of bi-encoders. The target model is a BERT-based dual-encoder (BERTDOT): query and passage are encoded independently, pooled into fixed-size vectors, and scored via dot product as in Eq. 2.11. The central challenge addressed by TAS-B is that stronger dense retrievers often require expensive training setups (e.g., repeated index refreshing for negative mining or very large batch sizes). TAS-B improves training efficiency by combining (i) *dual-teacher supervision* and (ii) *batch construction strategies* that make in-batch learning substantially more informative.

Dual-teacher supervision. Instead of relying on a single teacher, TAS-B uses two complementary teachers: a strong *pairwise* teacher that scores query–passage pairs with full cross-attention (cross-encoder style), and an *in-batch* teacher that better aligns with batch-wise contrastive training. Intuitively, the pairwise teacher provides precise supervision on which passage should outrank which, while the in-batch teacher encourages robust separation among many candidates encountered within a training batch. This combination aims to transfer accuracy from interaction-rich teachers into a fast bi-encoder student without changing the student architecture at inference time [45].

Topic Aware Sampling (TAS). A major bottleneck in in-batch training is that if a batch contains queries from unrelated topics, then the “in-batch negatives” are often trivially easy and contribute little learning signal. TAS addresses this by clustering training queries into topical groups before training (e.g., via k -means in an embedding space), and then sampling batches by drawing multiple queries from a small number of clusters. The goal is to concentrate semantically related queries in the same batch, so that passages associated with other queries become *harder* negatives. Formally, TAS clusters queries and composes a batch by sampling queries from sampled clusters:

$$\arg \min_C \sum_{i=1}^k \sum_{q \in C_i} \|q - v_i\|^2, \quad B = \{(q, p^+, p^-) \mid q \in \text{rand}(\text{rand}(C, n), \lfloor b/n \rfloor)\}, \quad (2.12)$$

where v_i denotes the centroid of cluster C_i , and b is the batch size [45].

Balanced margin sampling (TAS-Balanced). TAS-Balanced further refines batch construction by controlling the *difficulty* of training pairs. Given teacher scores, each query is associated with positive/negative passage pairs that have different score margins (easy negatives yield large margins and low information gain). TAS-Balanced samples pairs so that a batch covers a more balanced range of margins, reducing the dominance of overly easy negatives. A simple way to express this idea is to filter candidate pairs into margin ranges and sample uniformly across ranges:

$$H(P_q, i) = \{(p^+, p^-) \mid m_{\min} + i \cdot m \leq M_t(q, p^+) - M_t(q, p^-) < m_{\min} + (i + 1) \cdot m\}, \quad (2.13)$$

and then sample passage pairs for each query from a randomly chosen range. Combined with TAS, this produces batches that are simultaneously *topically coherent*

(harder in-batch competition) and *difficulty-balanced* (less wasted signal on trivial pairs). Figure 2.3 summarizes the difference between standard random batching, topic-aware batching (TAS), and the additional margin balancing step introduced in TAS-Balanced [45].

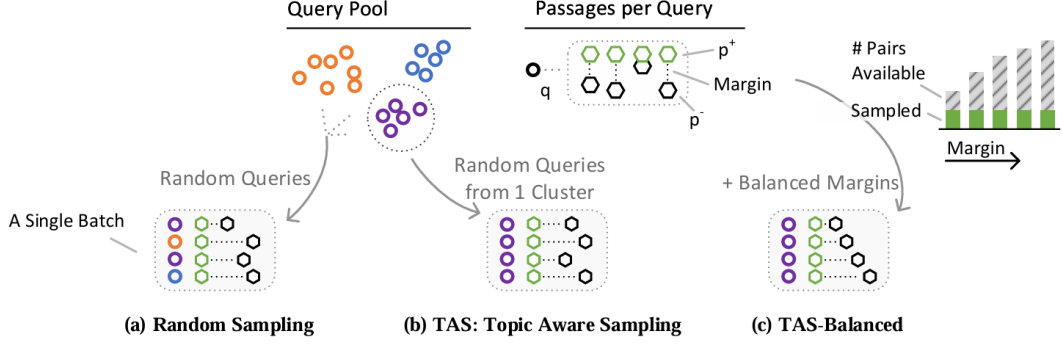


Figure 2.3: Batch construction strategies for dense retriever training: (a) random sampling often yields mostly easy in-batch negatives; (b) Topic Aware Sampling (TAS) increases topical competition by sampling queries from the same cluster; (c) TAS-Balanced further improves supervision by sampling training pairs across a balanced range of teacher-score margins [45].

Why TAS-B matters in practice. The outcome is a dense retriever that remains a standard single-vector bi-encoder at inference time (hence suitable for fast first-stage retrieval), but that is trained with substantially more informative batches and stronger supervision. This makes TAS-B a compelling baseline in realistic pipelines: it preserves the scalability advantages of dot-product retrieval while narrowing the effectiveness gap with heavier interaction-based models, and it does so with training setups that are accessible beyond large compute clusters [45].

2.4.3.2 Multiple representations

Single-vector bi-encoders compress an entire query or document into one embedding, which makes retrieval efficient but can be a bottleneck in expressiveness: fine-grained evidence (e.g., a single highly relevant phrase inside a long passage) may be diluted when everything is aggregated into a single point in the vector space. Multiple-representation systems address this limitation by encoding a text into *more than one* embedding, so that matching can be performed at a finer granularity while still avoiding the full cost of cross-encoders.

A common design choice is to keep the query representation compact, while allowing documents to expose richer sets of vectors. This family includes, among others, poly-encoders, COIL, and ColBERT [41, 46, 47]. Compared to single-vector systems, these models can better capture partial matches and local evidence, but they introduce new efficiency challenges: storing and searching multiple vectors per document increases index size and can complicate retrieval-time scoring and pruning.

Brief focus: ColBERT. ColBERT is a representative multiple-representation approach that combines contextualized encoders with an efficient *late interaction* mechanism (Figure 2.4) [41]. It encodes the query and the document *independently* into two sets (bags) of contextual token embeddings, shifting document encoding offline and amortizing it across queries. Relevance is then computed by aggregating token-level matches through a MaxSim operation: for each query embedding, it finds the most similar document embedding and sums these strongest matches across query terms.

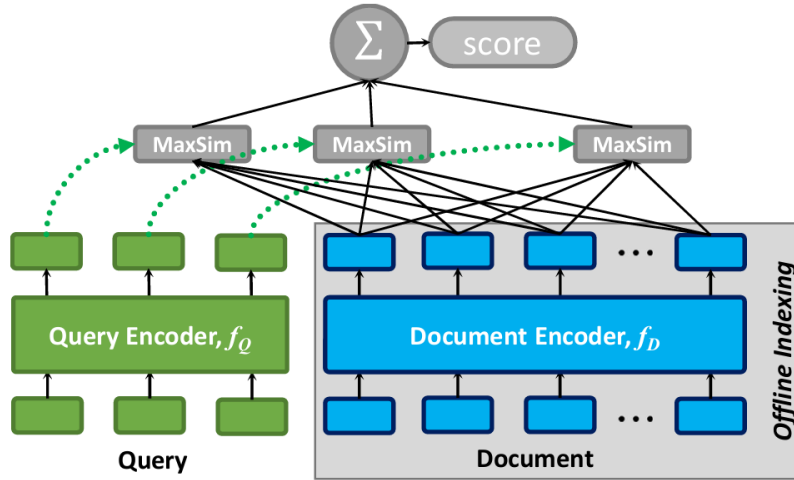


Figure 2.4: Overview of ColBERT: independent encoding of query and document into contextual token embeddings and late interaction scoring [41].

Concretely, if $E_q = \{\mathbf{q}_1, \dots, \mathbf{q}_m\}$ and $E_d = \{\mathbf{d}_1, \dots, \mathbf{d}_n\}$ denote the contextual token embeddings for query and document, ColBERT scores:

$$s(q, d) = \sum_{i=1}^m \max_{j \in \{1, \dots, n\}} \text{sim}(\mathbf{q}_i, \mathbf{d}_j), \quad (2.14)$$

where $\text{sim}(\cdot, \cdot)$ is typically a vector similarity (e.g., cosine). This late interaction preserves fine-grained matching signals while remaining much cheaper than all-

to-all cross-encoding, and it is designed to be compatible with vector-similarity indexing and pruning strategies for large-scale retrieval.

2.4.4 Evaluation of IR Systems

This section summarizes how IR systems are evaluated along two complementary axes: (i) *retrieval effectiveness*, i.e., how well a ranked list satisfies an information need; and (ii) *efficiency*, i.e., the computational cost required to produce that ranking. Finally, we briefly outline the benchmarking protocol typically used in IR experiments.

2.4.4.1 Retrieval effectiveness

Let q denote a query and let $\langle d_1, \dots, d_k \rangle$ be the top- k ranked documents returned by a system. We write $g_i \in \{0, 1, 2, \dots\}$ for the (graded) relevance label of d_i with respect to q , and $\text{rel}_i = \mathbb{I}[g_i > 0]$ for the corresponding binary indicator. Let Q be the set of evaluation queries. When needed, let R denote the (typically incomplete) number of relevant documents for q in the collection.

Precision@k and Recall@k. Precision and recall at cutoff k summarize, respectively, the *concentration* of relevant material in the top ranks and the *coverage* of relevant material within the inspected depth[2, 48]:

$$P@k = \frac{1}{k} \sum_{i=1}^k \text{rel}_i, \quad R@k = \frac{1}{R} \sum_{i=1}^k \text{rel}_i. \quad (2.15)$$

F1@k. When a single summary is useful, precision and recall can be combined through the harmonic mean:

$$F1@k = \frac{2 P@k R@k}{P@k + R@k}. \quad (2.16)$$

DCG@k and nDCG@k (graded relevance). Discounted Cumulative Gain rewards placing highly relevant items early in the ranking[5]:

$$DCG@k = \sum_{i=1}^k \frac{2^{g_i} - 1}{\log_2(i + 1)}. \quad (2.17)$$

To normalize across queries with different gain profiles, DCG is divided by the ideal DCG ($IDCG@k$), obtained by sorting the top- k documents by decreasing

relevance:

$$nDCG@k = \frac{DCG@k}{IDCG@k} \in [0, 1]. \quad (2.18)$$

Average Precision (AP) and MAP (binary relevance). Average Precision summarizes precision across the ranks where relevant documents appear:

$$AP = \frac{1}{R} \sum_{i=1}^N P@i \cdot \text{rel}_i, \quad (2.19)$$

where N is the depth of the ranked list (often $N = k$ in practice). Mean Average Precision is the average over queries:

$$MAP = \frac{1}{|Q|} \sum_{q \in Q} AP(q). \quad (2.20)$$

Reciprocal Rank and MRR (first relevant hit). Let rank_q be the rank position (1-based) of the first relevant document for query q ; if none is retrieved, set the reciprocal rank to 0 [49]. Then:

$$RR(q) = \frac{1}{\text{rank}_q}, \quad MRR = \frac{1}{|Q|} \sum_{q \in Q} RR(q). \quad (2.21)$$

ERR@k (optional, graded user model). Expected Reciprocal Rank models a cascade user who scans results top-down and stops when sufficiently satisfied:

$$ERR@k = \sum_{i=1}^k \frac{1}{i} P(\text{user stops at } i), \quad (2.22)$$

with a stopping probability defined from graded relevance (implementation depends on the chosen mapping from g_i to a satisfaction probability) [50].

Rank-Biased Precision. Under a persistence model where the user continues from rank i to $i + 1$ with probability p , Rank-Biased Precision at depth k is[51]:

$$RBP@k = (1 - p) \sum_{i=1}^k p^{i-1} \text{rel}_i. \quad (2.23)$$

2.4.4.2 Efficiency

Effectiveness must be interpreted together with efficiency, especially for first-stage retrieval [2, 3, 52]. We evaluate efficiency through latency, throughput, memory

footprint, and index build time [53].

Latency. Let $t_{\text{start}}^{(q)}$ and $t_{\text{end}}^{(q)}$ be the timestamps for processing query q end-to-end (encoding + ANN search + scoring). Query latency is:

$$\ell(q) = t_{\text{end}}^{(q)} - t_{\text{start}}^{(q)}. \quad (2.24)$$

We report mean latency to reflect the behavior.

Throughput. Given a batch of $|Q|$ queries processed in total time T , throughput (queries per second) is:

$$\text{QPS} = \frac{|Q|}{T}. \quad (2.25)$$

Memory footprint. Memory is measured as the storage required by the index and embeddings (and, when relevant, the in-memory resident set). A practical decomposition is:

$$M_{\text{total}} = M_{\text{index}} + M_{\text{doc_emb}} + M_{\text{model}} + M_{\text{overhead}}. \quad (2.26)$$

Index build time. Let T_{build} be the wall-clock time required to encode the corpus and construct the ANN index:

$$T_{\text{build}} = T_{\text{encode}} + T_{\text{construct}}. \quad (2.27)$$

This cost is amortized over many queries but is critical when indices must be refreshed.

2.5 Quality Estimation for Retrieval: QualT5

Neural retrieval pipelines are typically designed under the assumption that the corpus is fixed: the system specifies how to index and rank documents, but not whether all passages are worth the computational and storage costs. In practice, large passage collections often contain segments that are unlikely to satisfy *any* plausible information need (e.g., boilerplate, thin or highly context-dependent fragments). Indexing and searching such passages can be wasteful, increasing memory footprint, indexing time, and query-time costs without providing corresponding gains in effectiveness. QualT5 addresses this problem by introducing a

query-agnostic notion of passage quality and using it as a signal for *static pruning*, i.e., removing low-quality passages before indexing [54, 55].

2.5.1 From query-conditioned relevance to query-agnostic quality

Traditional neural relevance models estimate the relevance of a passage p with respect to a specific query q , i.e. a query-conditioned score:

$$\text{Rel}(p \mid q), \quad (2.28)$$

where $q \in Q$ is drawn from the space of user queries and $p \in P$ is drawn from the corpus of passages. In this setting, relevance is meaningful only relative to a query and to the competing candidates ranked for that query.

QualT5 departs from this paradigm by modeling a passage-level *quality* signal:

$$\text{Qual}(p), \quad (2.29)$$

intended to capture whether a passage is *generally worth indexing*, i.e., potentially useful for *any* query in the anticipated workload (Figure 2.5). Conceptually, relevance is “local” to a query, while quality is “global” with respect to the (unknown) query distribution. This reframes part of retrieval efficiency as a corpus ingestion decision: rather than ranking everything, the system first learns which content is worth keeping in the searchable collection [54].

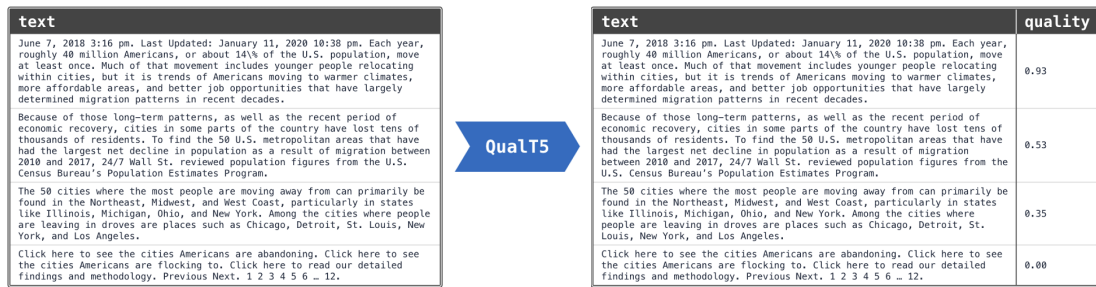


Figure 2.5: QualT5- quality estimation of the passages [54].

2.5.2 QualT5 as a supervised passage quality estimator

QualT5 instantiates $\text{Qual}(\cdot)$ using a T5 encoder–decoder model T5[23] fine-tuned to produce a binary decision about passage usefulness. Since T5 follows a text-to-text interface, the input passage is wrapped in a short prompt and the model

is trained to generate a discrete label (e.g., **true/false**) indicating whether the passage should be treated as high quality [54].

Fine-tuning as text-to-text classification (query ignored). A key aspect is that supervision can be derived from standard IR relevance data (e.g., triples containing a query, a relevant passage, and a non-relevant passage), but the quality model *ignores the query* during training. Let $(q, p^+, p^-) \in T$ denote a training triple where p^+ is judged relevant to q and p^- is judged non-relevant. QualT5 converts this into *passage-only* labels: $p^+ \mapsto 1$ and $p^- \mapsto 0$, and fine-tunes a passage predictor $M_\theta(p)$ using pointwise supervision:

$$\arg \min_{\theta} \mathcal{L}(M_\theta(p^+), 1) + \mathcal{L}(M_\theta(p^-), 0), \quad \forall (q, p^+, p^-) \in T, \quad (2.30)$$

where $\mathcal{L}$ is a classification loss (e.g., cross-entropy) and θ are the model parameters. Intuitively, although labels originate from query-specific assessments, repeated exposure to positives vs. negatives allows the model to learn corpus-level textual cues correlated with general usefulness for retrieval.

Training details (practical considerations). In the paper, fine-tuning is performed on MS MARCO [56] training triples with an Adam optimizer [57] and a learning rate of $5 \cdot 10^{-5}$, for 10k iterations with batch size 16. This comparatively short schedule is motivated by overfitting effects observed when training for too long on MS MARCO triples, and it reflects the fact that the goal is not to memorize query-specific relevance, but to distill a stable passage-level quality signal.

2.5.3 Static pruning with a quality threshold

Once a quality estimator is available, it can be applied *offline* to every passage in the corpus. Given a threshold t , static pruning retains only passages whose estimated quality exceeds t :

$$P' = \{p \mid p \in P \wedge \text{Qual}(p) \geq t\}. \quad (2.31)$$

The pruned corpus P' is then indexed and used by downstream retrieval pipelines. The threshold can be chosen to match an operational target (e.g., “prune 20% of passages”) or tuned empirically to preserve effectiveness while reducing costs. Expected benefits include lower indexing cost, reduced storage/memory footprint

(especially for dense vector indexes), and improved query-time efficiency due to smaller candidate spaces [54, 55].

2.5.4 Model size and efficiency trade-offs

A practical advantage of using T5 is the availability of multiple model sizes (e.g., Tiny/Small/Base), enabling a direct trade-off between the accuracy of quality estimation and the cost of scoring passages offline. The paper highlights that smaller variants substantially reduce per-passage inference latency, making the pruning step more compatible with large-scale indexing and allowing the pruning procedure to “break even” when a moderate fraction of the corpus is removed.

2.5.5 Positioning within retrieval pipelines

QualT5 does not replace retrieval models; it acts *upstream* of indexing as a corpus filtering mechanism. In a multi-stage pipeline, it can be interpreted as a one-shot gate applied at ingestion time: first prune low-quality passages, then build lexical/dense/sparse indices on the remaining set, and finally apply standard retrieval and re-ranking steps on a smaller (and potentially cleaner) corpus. This perspective reframes part of the efficiency problem of Neural IR from “how to retrieve faster” to “what content is worth retrieving from” in the first place.

2.6 Similarity Search and Indexing with FAISS

Neural retrieval systems often reduce first-stage retrieval to a *nearest-neighbor* problem in a vector space: each passage (or document) is represented by an embedding $\mathbf{x} \in \mathbb{R}^d$, the query is represented by $\mathbf{q} \in \mathbb{R}^d$, and retrieval returns the top- k items that are closest to $\mathbf{q}$ under a chosen similarity. This section summarizes the main indexing families used for similarity search and clarifies the key trade-offs between accuracy, latency, memory footprint, and index construction cost [53, 58].

Similarity measures. Given a database of vectors $\{\mathbf{x}_i\}_{i=1}^N$, the k -NN search can be written as:

$$(n_1, \dots, n_k) = \arg \min_{i=1 \dots N}^k \|\mathbf{q} - \mathbf{x}_i\|_2, \quad (2.32)$$

for Euclidean distance, or as an $\arg \max$ for inner product:

$$(n_1, \dots, n_k) = \arg \max_{i=1 \dots N}^k \mathbf{q}^\top \mathbf{x}_i. \quad (2.33)$$

Cosine similarity is typically implemented by L_2 -normalizing vectors so that cosine reduces to inner product [58].

Two complementary levers. Most scalable indexes combine two ideas: (i) *candidate pruning*, i.e., reducing how many vectors are compared with the query, and (ii) *compression*, i.e., storing approximate vector representations to reduce memory and speed up scoring. The following subsections detail four common families that instantiate these levers in different ways [53, 59].

2.6.1 Exact search (brute force)

The simplest baseline is *exhaustive search*: compute the similarity between $\mathbf{q}$ and all database vectors $\mathbf{x}_1, \dots, \mathbf{x}_N$ and return the top- k [53, 58]. Its query-time cost scales linearly with N :

$$T_{\text{scan}}(q) \propto N \cdot d, \quad (2.34)$$

since each score requires processing d dimensions (for dot product or L_2 distance).

Compute/memory trade-off. Exhaustive search yields maximal accuracy because it performs no approximation; however, it requires storing all vectors (typically as floating-point arrays) and performs N comparisons per query. It is therefore most attractive when: (i) the corpus is small to medium, (ii) a high-end accelerator is available for fast matrix-style distance computation, or (iii) exactness is required for validation and ablation purposes.

2.6.2 Inverted files (IVF)

Inverted-file (IVF) indexing is a *non-exhaustive* nearest-neighbor search strategy that accelerates retrieval by dramatically reducing the number of database vectors compared against a query. The core idea is to *partition* the embedding space into K regions using a *coarse quantizer*, and at query time to scan only the P most promising regions (controlled by `nprobe`). This turns the k -NN problem from “compare against everything” into “compare against candidates from a small set of selected lists” [53, 58].

Index construction: coarse quantization and inverted lists

Let $\mathcal{X} = \{\mathbf{x}_i\}_{i=1}^N$ be a database of vectors in $\mathbb{R}^d$. IVF learns a set of coarse centroids $\mathcal{M} = \{\boldsymbol{\mu}_1, \dots, \boldsymbol{\mu}_K\}$ (typically via k -means) [58, 59]. The coarse quantizer defines

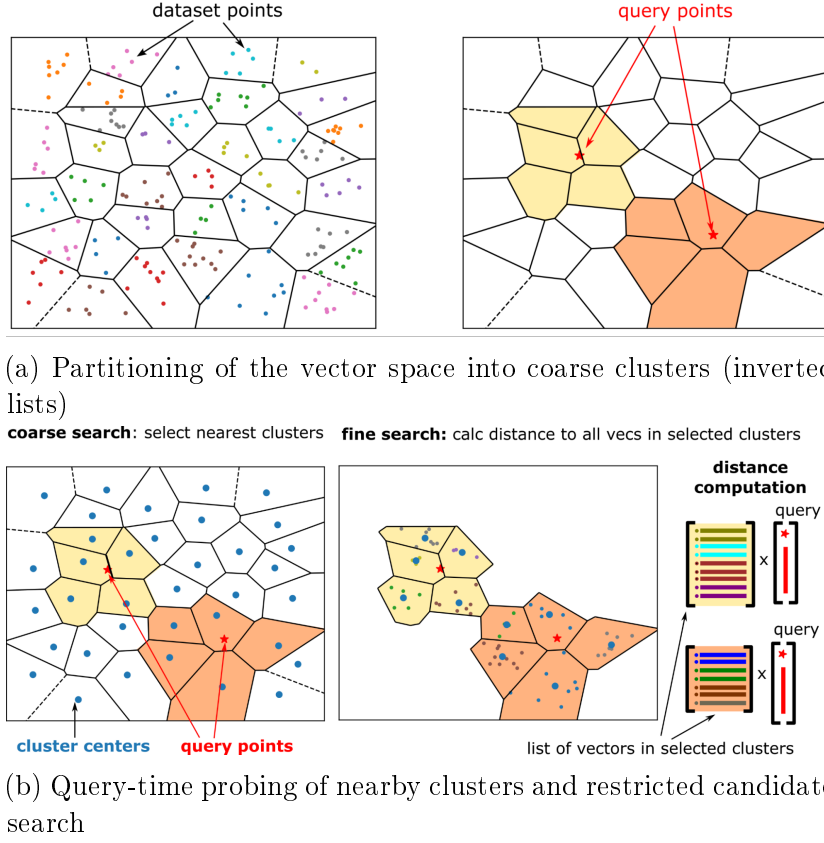


Figure 2.6: Illustration of inverted-file (IVF) indexing. The dataset is partitioned into clusters during indexing, and at query time the search is restricted to a subset of nearby clusters determined by the query [60].

an assignment function that maps each database vector to its nearest centroid:

$$q(\mathbf{x}) = \arg \min_{j \in \{1, \dots, K\}} \|\mathbf{x} - \boldsymbol{\mu}_j\|_2. \quad (2.35)$$

Each centroid corresponds to an *inverted list* (or *posting list* in vector space) that stores all vectors assigned to that centroid, together with their identifiers. Intuitively, IVF builds a lightweight “routing layer” over the database: centroids summarize regions of the space, and the inverted lists store the actual candidates that live in each region.

From a storage perspective, an IVF index contains: (i) the coarse centroids (the quantizer model), and (ii) the per-list data structure holding document IDs and their vector representations (potentially in full precision, or in a compressed form if combined with quantization). IVF on its own is primarily an acceleration mechanism through *candidate pruning*: it reduces the number of distances computed at query time by restricting the scan to a subset of lists [53].

Query processing: list selection (nprobe) and candidate scanning

Given a query embedding $\mathbf{q}$, IVF search proceeds in two stages:

1) Coarse search (selecting lists). The system computes distances between $\mathbf{q}$ and the K centroids and selects the P closest centroids:

$$\mathcal{P}(\mathbf{q}) = \text{TopP } j \in \{1, \dots, K\} (-\|\mathbf{q} - \boldsymbol{\mu}_j\|_2), \quad (2.36)$$

where P is `nprobe`. This step decides *which regions of the embedding space* are considered relevant for the query.

2) Candidate scan and top- k selection. The system scans the vectors stored in the inverted lists indexed by $\mathcal{P}(\mathbf{q})$, computes the chosen similarity (e.g., inner product or L_2 distance) against each candidate, and maintains a top- k structure (e.g., a heap) to return the best scoring items. The search is non-exhaustive because the scan is restricted to candidates contained in only P lists.

Key hyperparameters: `nlist` and `nprobe`

IVF exposes two primary knobs that define the accuracy–efficiency operating point:

- `nlist` (K): number of coarse clusters / inverted lists (partition granularity).
- `nprobe` (P): number of lists visited per query (search breadth).

Increasing `nprobe` typically increases recall because more candidates are examined, but it also increases latency and reduces throughput. Increasing `nlist` makes lists smaller on average (which can reduce scan cost), but it increases the complexity of the coarse partition (training and/or searching among more centroids) and may exacerbate list imbalance [58].

A simple complexity intuition (and why it is only a first approximation)

A useful back-of-the-envelope decomposition of the per-query work is: (i) coarse search over K centroids, and (ii) candidate scanning over P selected lists.

In an idealized setting where: (1) coarse search evaluates all centroid distances, (2) lists are perfectly balanced, and (3) candidates are stored in full precision, the expected number of distance computations can be approximated as:

$$N_{\text{dist}} \approx K + P \cdot \frac{N}{K}. \quad (2.37)$$

The first term grows with `nlist`, while the second term decreases with `nlist` but grows linearly with `nprobe`. Minimizing (2.37) with respect to K (for fixed P) yields the classical rule-of-thumb:

$$K^* \approx \sqrt{PN}, \quad (2.38)$$

which provides an order-of-magnitude intuition for selecting `nlist`.

In practice, however, (2.37)–(2.38) should be treated as guidance rather than as a prescription:

(i) `nprobe` and `nlist` interact under a fixed quality target. When K becomes large (finer partitioning), true nearest neighbors may lie in adjacent regions; maintaining the same recall often requires increasing P as well. Thus, K and P are not independent when one targets a given effectiveness level.

(ii) List imbalance increases the scan cost. Real datasets rarely yield uniform list sizes. If some lists are much larger than others, queries that probe those large lists incur higher candidate scanning cost, increasing average latency.

(iii) Coarse search may itself be approximate. For very large K , computing distances to all centroids can become a non-trivial overhead. In realistic settings, the centroid search can be accelerated with approximate methods, which reduce coarse-search time at the cost of occasionally probing suboptimal lists (and potentially losing recall).

Experimental tuning perspective: why sweeps over `nprobe` are natural

A practical way to characterize IVF is to view `nprobe` as a *search-time* parameter with a largely monotone effect: higher `nprobe` increases candidate coverage and tends to improve recall, while reducing speed. This makes it natural to: (i) choose one or a few plausible values of `nlist`, (ii) sweep `nprobe` over a range, (iii) report and analyze the resulting speed–quality frontier.

Beyond heuristics, the selection of `nlist`/`nprobe` should be motivated by: (1) the geometry of the embedding space, (2) the empirical balance of list sizes, (3) the overhead of coarse search as K grows, and (4) the recall level required at the cutoff that matters for the downstream pipeline (e.g., candidate coverage for later stages) [53, 58].

2.6.3 Graph-based methods (HNSW)

Graph-based indexes accelerate search by building a sparse proximity graph over vectors [61]. Each vector is a node, edges connect it to a small set of neighbors, and query-time search traverses the graph by repeatedly expanding promising candidates (best-first exploration) until a stopping criterion is met.

Hierarchical navigation. HNSW introduces multiple graph layers. Upper layers provide long-range “express” connections that quickly move the search towards the relevant region, while lower layers refine the search locally. This hierarchical structure tends to produce strong accuracy–efficiency trade-offs with relatively simple query-time control [61].

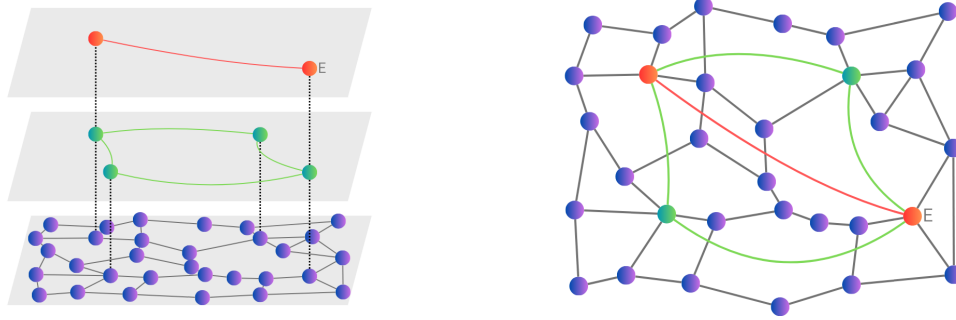


Figure 2.7: Left: illustration of an HNSW graph with three connection lengths—short links in grey, longer links in green, and the longest links in red. E denotes the entry point. Right: the same HNSW represented as a stack of three layers; dotted lines indicate the corresponding position of each vector in the layer below [62].

Key parameters: efConstruction and efSearch. Two parameters are typically used to tune HNSW:

- **efConstruction** controls how thoroughly neighbors are explored during index building. Higher values increase build time and memory overhead but usually improve search quality.
- **efSearch** controls how many candidates are explored at query time. Higher values increase recall but reduce throughput.

Graph methods generally trade memory for speed and recall: storing neighbor links adds overhead beyond the raw vector storage, but enables efficient navigation without scanning large candidate sets [63].

2.6.4 Compression and quantization (PQ/OPQ)

When storing and scanning full-precision vectors is too expensive, *quantization* compresses vectors into compact codes and uses approximate distance computations. This reduces memory footprint and often accelerates scoring, at the cost of some accuracy [58, 59].

Product Quantization (PQ). Product Quantization splits each vector into m sub-vectors and quantizes each part independently:

$$\mathbf{x} = [\mathbf{x}_1 | \cdots | \mathbf{x}_m], \quad \hat{\mathbf{x}} = [q_1(\mathbf{x}_1) | \cdots | q_m(\mathbf{x}_m)], \quad (2.39)$$

where each q_j maps a sub-vector to its nearest centroid in a sub-codebook. The stored representation is then the sequence of codebook indices rather than the original float values, yielding large memory savings. Approximate distances can be computed efficiently using precomputed lookup tables for each sub-quantizer [59].

OPQ (Optimized Product Quantization). A limitation of PQ is that fixed sub-vector boundaries may not align with the geometry of the embedding space. OPQ improves PQ by learning a linear rotation (and, optionally, a dimensionality reduction) so that information is distributed more evenly across subspaces before quantization. The code length remains unchanged, but quantization error is typically reduced [64].

Combining pruning and compression: IVF-PQ (residual quantization).

In large-scale settings, pruning and compression are often combined: a coarse partitioning strategy reduces the number of candidates that must be evaluated, while quantization reduces the cost per candidate and the memory footprint. A standard instantiation of this idea is *IVF-PQ*, where a coarse quantizer partitions the dataset into inverted lists and, within each list, the *residual* $\mathbf{r} = \mathbf{x} - \boldsymbol{\mu}_{q(\mathbf{x})}$ between a vector and its list centroid is PQ-encoded rather than storing the full-precision vector. At query time, the system probes only a subset of lists (**nprobe**) and scans their candidates using compact PQ codes; approximate distance computation is performed efficiently via PQ lookup tables (ADC), which can be interpreted as a coarse centroid term plus a residual term approximated through the PQ code. This yields a flexible design space where recall, latency, and memory can be tuned jointly by adjusting probing parameters and code size [53, 58, 59].

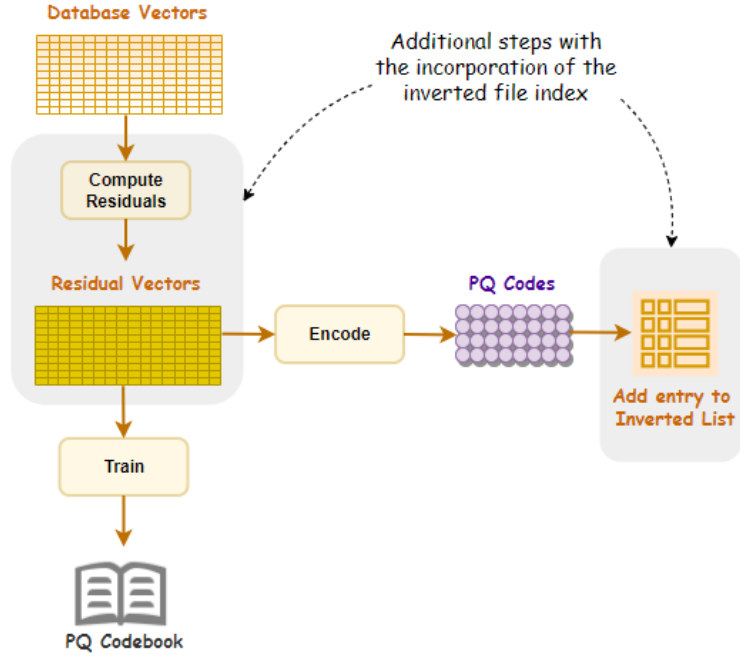


Figure 2.8: IVF-PQ indexing pipeline: vectors are assigned to IVF inverted lists, residuals w.r.t. the list centroid are computed and PQ-encoded, and the resulting codes are stored in the corresponding inverted lists [65].

Summary. Exact scan provides the strongest accuracy but does not scale in N ; IVF accelerates search via list probing; HNSW accelerates search via graph navigation with tunable exploration effort; PQ/OPQ reduce memory and scoring cost through vector compression. Together, these methods define a set of practical operating points for neural retrieval, where the choice of index reflects the desired balance among recall, latency, throughput, and hardware constraints.

Chapter 3

Research Objectives

3.1 Problem setting: quality-aware corpus partitioning for efficient retrieval

Neural first-stage retrieval on large passage collections is shaped by a recurring practical tension between *effectiveness* and *efficiency*. Searching the entire corpus offers the strongest form of coverage: in principle, every relevant passage remains accessible. However, this choice comes with a clear cost in storage and query-time, especially when the collection scales to millions of items. At the opposite end, reducing the corpus can make retrieval faster and lighter, but it may also remove passages that matter precisely because they are rare, long-tail, or hard to predict in advance.

This thesis ¹ explores a middle ground between these two extremes. Rather than treating corpus reduction as a one-way decision, we ask whether the collection can be *organized* so that retrieval is fast for the majority of queries while still remaining robust when extra coverage is needed. The key idea is to separate passages into two groups: a *high-quality* tier, intended to contain the most informative and consistently useful content, and a *low-quality* tier that is kept as a safety net. Under this view, most queries should be answerable by searching the high tier only, whereas the low tier is consulted selectively for the smaller set of queries for which the high tier appears insufficient. In this design, the high-quality tier is intentionally kept lightweight so it can act as the default, fast search space. Conversely, the low-quality tier is allowed to be larger and “heavier”, because it is accessed only occasionally when a query requires additional coverage.

Two implications follow from this design. First, efficiency improvements are

¹Code and experimental scripts are available at: github.com/comocomo39/TwoTierArchitecture.

pursued by concentrating routine search effort on a smaller, more informative pool rather than by permanently discarding data. Second, effectiveness is protected because the remaining portion of the corpus is not removed, but remains available as a fallback. The research challenge, therefore, is not whether to prune, but how to partition and exploit the partition in a principled and measurable way.

To make this idea operational, the thesis relies on an automatic notion of passage “quality” that can be assigned consistently across the entire collection. The specific scoring approach, the way thresholds are selected, and how the resulting tiers are instantiated in the retrieval pipeline are described in Chapter 4 and in the dedicated research-question chapters. Here, we focus on the objectives and the questions that guide the experimental study.

3.2 Research Questions and Experimental Analyses

The experimental study is structured around three research questions (RQs), each corresponding to a core design choice: (i) the *quality signal* used to induce the partition, (ii) the *retrieval architecture* used to exploit the two-tier partition, and (iii) the resulting *effectiveness–efficiency trade-off* across partition thresholds and search effort. Implementation details and the full evaluation protocol are described in Chapter 4 and in the dedicated RQ chapters; here we state the RQs and summarize the analyses conducted to answer them.

3.2.1 RQ1: QualT5 quality signal—distribution, separability, and tiny vs base

RQ1. *How do QualT5-tiny and QualT5-base differ in their quality-score distributions, and how reliably do they induce a high/low partition of the corpus that is stable and interpretable?*

RQ1 examines whether the quality estimator provides a robust and usable signal for tiering. We study the empirical distributions of QualT5 scores, compare how sharply the two variants separate the corpus at matched thresholds, and quantify agreement between the induced high-tier sets. When relevant, we also check whether the partition exhibits systematic instability across subsets of the collection, to identify threshold regimes that support meaningful and reproducible tiered experiments.

3.2.2 RQ2: Architectural design—Two-Tier vs Virtual Partitioning

RQ2. *What are the practical differences between a Two-Tier design (separate high/low indexes) and Virtual Partitioning (a single index with logical filtering), and how do these architectural choices affect latency, and fallback behavior under comparable retrieval settings?*

RQ2 focuses on *architectural* effects: given the same quality-induced partition and comparable ANN search settings, do the two execution designs exhibit different latency profiles, and fallback dynamics? Concretely, we analyze (i) average latency, and (ii) fallback rate and additional work introduced by the low-tier access. The goal is to isolate how the chosen execution design influences cost and control at query time, before comparing end-to-end trade-offs in RQ3.

3.2.3 RQ3: Final performance—effectiveness vs efficiency across partition thresholds

RQ3. *Across different high/low split thresholds, how do Two-Tier and Virtual Partitioning trade off effectiveness (e.g., $NDCG@10$, $MRR@10$) and efficiency (average latency), and how do they compare to a full-index baseline with no tiering?*

RQ3 integrates the architectural choices from RQ2 with the quality-signal regimes identified in RQ1. We sweep multiple partition thresholds (top- $p\%$ retained in the high tier) and evaluate both architectures under consistent retrieval settings, comparing them against a full-index baseline to quantify the net impact of tiering. Results are reported as effectiveness–efficiency operating curves, highlighting where tiered retrieval preserves baseline-level effectiveness while reducing query-time. The outcome of RQ3 is a set of practical operating points and a clear account of when tiering is beneficial under realistic efficiency constraints.

Chapter 4

Methodology

4.1 Method overview

Dense first-stage retrieval can be framed as a nearest-neighbor search problem in a vector space: each query is mapped to an embedding and the system returns the top- k passages whose embeddings are most similar. At realistic collection scale, however, efficiency depends not only on the index and the search algorithm, but also on the *composition of the candidate pool*: large passage corpora contain substantial amounts of content that is repetitive, noisy, poorly structured, or otherwise unlikely to be useful as first-stage candidates. Indexing and searching this long tail can impose a significant computational burden while contributing marginal utility to the top- k set.

This thesis adopts the perspective that a query-independent notion of passage quality can be used as an *indexing prior* to restructure the first-stage search space. The central idea is to assign each passage a scalar quality score computed offline, then use that score to induce a family of partitions under explicit percentage thresholds (*high-tier share*). For a given *high-tier share*, passages are separated into a *high-quality* subset (the default search space) and a complementary *low-quality* subset (a recovery space). The purpose is not to claim that the low-quality subset is never relevant, but rather that it can be consulted *selectively* to reduce average cost without systematically harming retrieval behavior.

Selectivity is crucial. If both subsets are always searched, no efficiency benefit is obtained; if only the high-quality subset is searched, relevant passages assigned to the low tier may be missed. To resolve this tension, the methodology implements a two-step execution pattern: (i) search the high tier by default and inspect the high-tier output, (ii) trigger a low-tier fallback only when the high-tier output indicates uncertainty. This yields an explicit trade-off between efficiency (how

often the low tier is avoided) and robustness (how often the system recovers from low-tier passages).

Two architectural realizations of this idea are studied. The first (*Two-Tier*) materializes the split as two separate retrieval structures. The second (*Virtual Partitioning*) maintains a single retrieval structure and enforces tier membership through logical filtering at query time. This separation isolates a methodological question: whether the benefits of tiered retrieval come mainly from physically separating the indexes, or can be achieved through logical partitioning while keeping a single index.

Concrete design choices (datasets, model instantiations, indexing hyperparameters, sweep grids, metrics, and timing protocol) are reported in Chapter 5. Here we define the approach at the level of pipeline logic and execution semantics.

4.2 Problem formulation

Let $\mathcal{P}$ be a passage collection and q a query. Dense retrieval maps text into a shared vector space through an encoder $\phi(\cdot)$: $\mathbf{q} = \phi(q)$ and $\mathbf{x}_p = \phi(p)$. Given a similarity function $s(\cdot, \cdot)$, retrieval returns the top- k passages from a candidate set $\mathcal{S} \subseteq \mathcal{P}$:

$$\text{TopK}(q; \mathcal{S}) = \arg \max_{p \in \mathcal{S}}^k s(\mathbf{q}, \mathbf{x}_p). \quad (4.1)$$

Each passage p is also assigned a query-independent quality score $Q(p) \in \mathbb{R}$ computed offline. Let $\alpha \in (0, 1]$ denote the *high-tier share*, i.e., the fraction of the collection retained in the high-quality tier (we report splits as percentages $p\%$, where $p = 100 * \alpha$). We define the two tiers as:

$$H_\alpha = \{p \in \mathcal{P} : Q(p) \text{ is within the top } \alpha \text{ fraction}\}, \quad L_\alpha = \mathcal{P} \setminus H_\alpha. \quad (4.2)$$

Smaller α yields a smaller default search space (higher efficiency potential) but increases the probability that relevant passages fall in L_α ; larger α reduces that risk while shrinking the efficiency opportunity.

4.3 Index construction paradigms

The partition induced by $Q(p)$ can be operationalized in two different ways.

4.3.1 Two-Tier index construction

Two-Tier materializes the split by building two separate retrieval structures: one over H_α (high index) and one over L_α (low index). Figure 4.1 illustrates the process: passages are scored, ordered by score, then partitioned to form the two physical indexes.

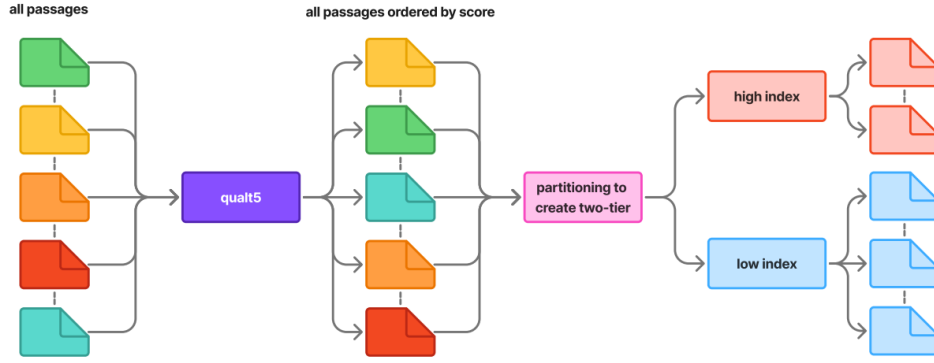


Figure 4.1: Two-Tier index creation: passages are scored offline, ordered by quality, then partitioned into a high tier and a low tier, resulting in two physical indexes.

4.3.2 Virtual Partitioning index construction

Virtual Partitioning keeps a single physical retrieval structure built over the full collection, and uses quality scores to define *virtual* high/low partitions inside the same index. Figure 4.2 depicts the idea: passages are scored, indexed together, and tier membership is later enforced via filtering.

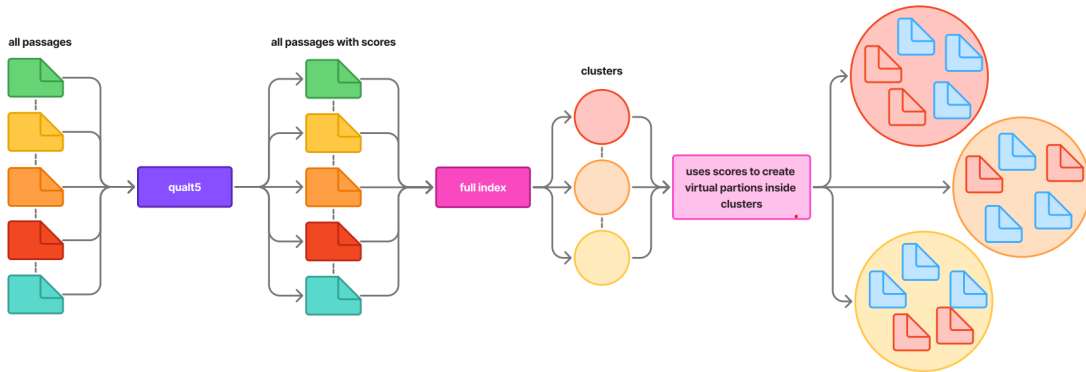


Figure 4.2: Virtual Partitioning index creation: passages are scored offline, indexed in a single full index, and quality scores define virtual high/low partitions (enforced at query time via filtering).

4.4 Retrieval architectures and baseline

4.4.1 Baseline

The baseline corresponds to a standard dense first-stage retriever that searches the full set $\mathcal{P}$ in a single pass and returns $\text{TopK}(q; \mathcal{P})$. It provides the reference ranking behavior and cost against which tiered execution is compared.

4.4.2 Two-Tier execution

In Two-Tier execution, the system runs a first retrieval pass over H_α . A fallback policy then decides whether to run a second pass over L_α . If fallback is activated, candidates from both passes are merged by score and truncated to top- k ; otherwise, the system returns high-tier results only. When fallback is triggered, candidates from the two passes are combined and re-ranked using the same similarity function $s(\mathbf{q}, \mathbf{x}_p)$. Since both passes use the same encoder and scoring function, scores are directly comparable and the final output is the global top- k over $H_\alpha \cup L_\alpha$.

4.4.3 Virtual Partitioning execution

In Virtual Partitioning execution, the system runs the first pass while restricting eligibility to H_α within a single index. If fallback is activated, it runs a second pass under the complementary eligibility constraint for L_α and merges candidates by score. A key methodological property (highlighted in Figure 4.4b) is that the second pass can reuse the same coarse routing assignments computed for the first pass, reducing duplicated work compared to naively running two independent searches.

Coarse assignment reuse across tiers. The two architectures differ in whether the high- and low-tier passes can reuse the same IVF coarse assignment. In our Two-Tier setup, the high and low indexes do not share the same quantizer, therefore coarse assignments are not reused across tiers. In Virtual Partitioning, both passes query the same physical index and share the same clusters, so the coarse assignment computed for the high-tier pass can be reused for the low-tier pass.

4.5 Gating and query-time outcomes

Tiering provides efficiency only if the low tier is not systematically queried. Therefore, after the high-tier pass the system computes an uncertainty signal from the high-tier output and applies a gating rule: if the query is deemed “certain”, the system stops; if “uncertain”, it triggers a low-tier pass and then merges results. In practice, uncertainty is computed from the score distribution returned by the high-tier pass (e.g., margin- or dispersion-based signals computed on the top- k list). The specific signal and its operating threshold are treated as experimental parameters and are defined and analyzed in RQ2.

Figure 4.3 shows the closed-gate case: fallback is not activated and the system returns the high-tier top- k only.

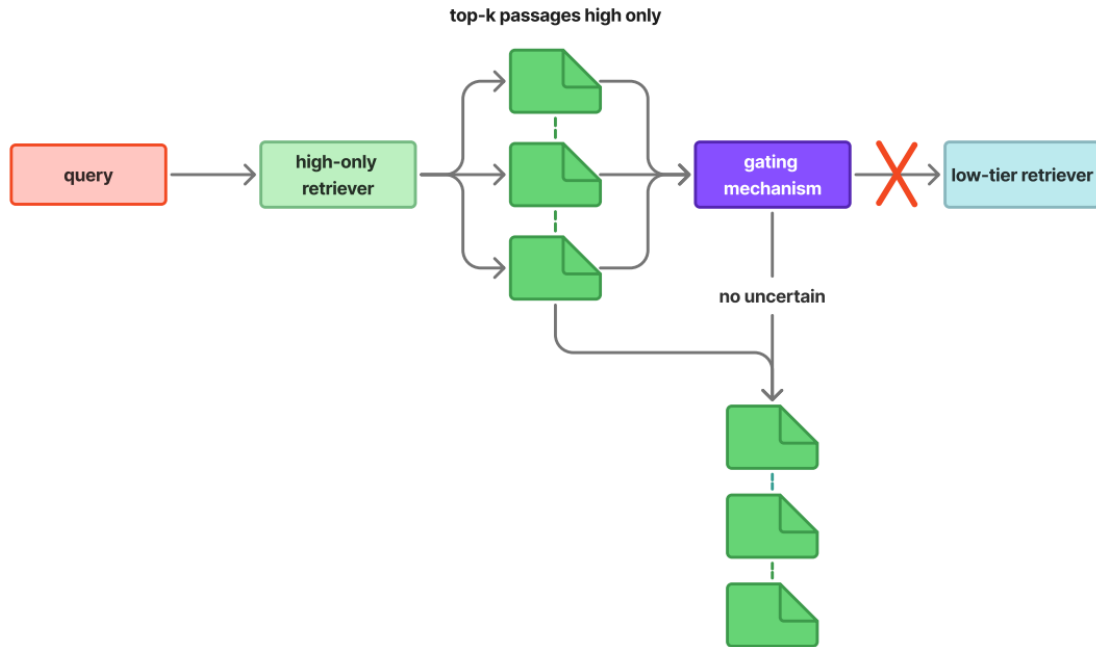
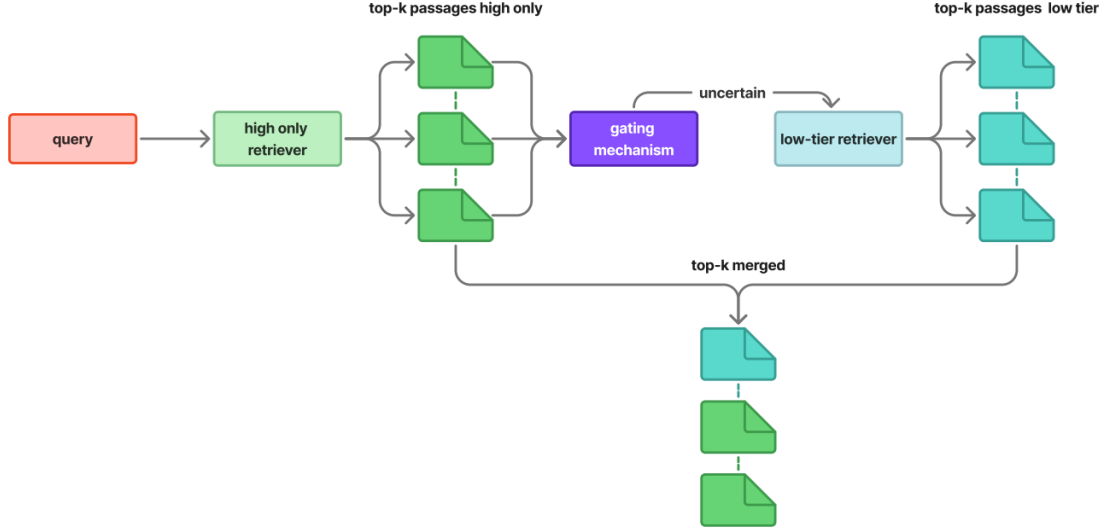
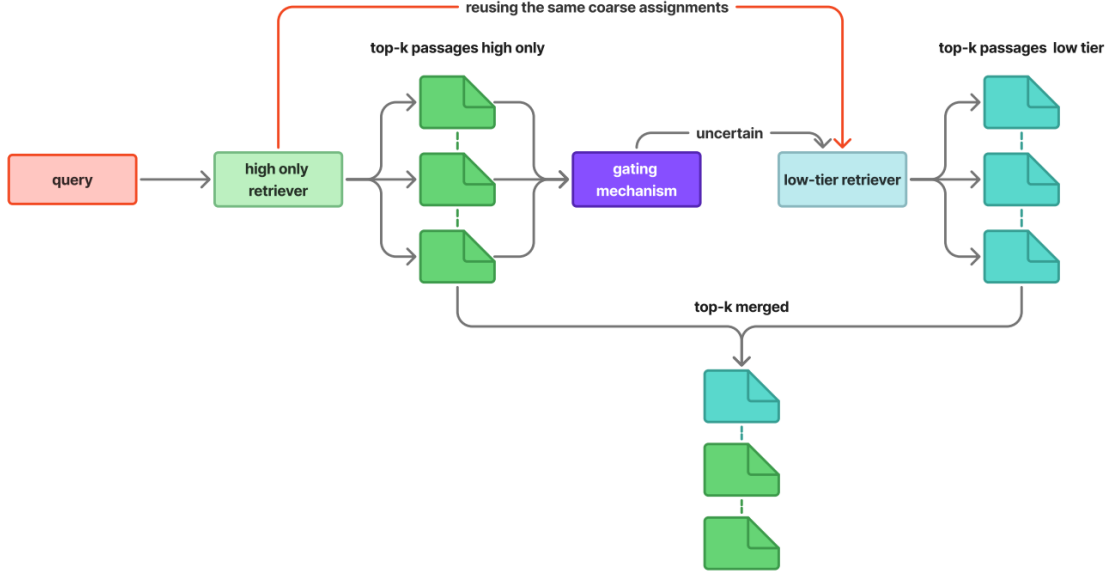


Figure 4.3: Closed-gate outcome: the gating mechanism does not activate fallback, so retrieval terminates after the high-tier pass and returns high-tier top- k results only.

When fallback is activated, the query-time behavior depends on the architecture. Figure 4.4a shows Two-Tier fallback (second pass on the low-tier retriever). Figure 4.4b shows Virtual Partitioning fallback; in this case the second pass can reuse the same coarse assignments computed for the first pass, minimizing redundant routing work.



(a) Two-Tier fallback: a second pass is executed on the low-tier retriever and merged with high-tier candidates.



(b) Virtual Partitioning fallback: a second pass is executed under low-tier filtering, reusing the same coarse assignments from the high-tier pass.

Figure 4.4: Open-gate outcomes: when uncertainty is detected, the system triggers a low-tier pass and merges candidates to produce the final top- k .

Chapter 5

Experimental Setup

This chapter defines the experimental setup adopted throughout the thesis, including the data and evaluation protocol, the retrieval and indexing configuration, the main implementation choices, and the metrics and timing procedure. The goal is to ensure that all comparisons are fair, directly comparable, and reproducible.

5.1 Dataset

All experiments are conducted on the MS MARCO Passage Ranking benchmark through the `ir_datasets` interface. We index the full passage collection `msmarco-passage`, which contains 8,841,823 passages. Effectiveness is evaluated on the official development *small* subset, `msmarco-passage/dev/small`, consisting of 6,980 natural-language queries with 7,437 relevance judgments (qrels). This split is a curated subset of the full development set and is commonly adopted to speed up experimental iteration while retaining the official evaluation protocol. Both query topics and qrels are retrieved from the same `ir_datasets` package to guarantee consistent identifiers and full reproducibility across all configurations [56, 66].

5.2 Quality score models

To induce the high/low partition of the corpus, we employ QualT5 as an automatic passage-quality estimator and compare two variants: QUALT5-TINY (15.6M parameters) [67] and QUALT5-BASE (0.2B parameters) [68]. Quality scores are computed offline and cached [69, 70], enabling controlled comparisons where the retrieval and indexing components remain fixed while only the quality signal (tiny vs. base) changes [71].

5.3 Dense representations

Dense retrieval operates over vector representations produced by a dual-encoder (TAS-B), which embeds both passages and queries into a shared vector space. All passages are encoded with TAS-B into fixed-dimensional vectors ($d = 768$) [45, 72]. To avoid repeated encoding across configurations, passage embeddings are pre-computed once and cached on disk. Query embeddings are computed with the same encoder at evaluation time and provided to the ANN searchers in batch form.

5.4 Tooling

The experimental pipeline is implemented in Python and builds on: (i) FAISS for ANN indexing and search [73], (ii) PyTerrier for retrieval pipeline orchestration and evaluation [74], (iii) HuggingFace for model loading and inference [67, 68], and (iv) standard numerical libraries (NumPy/Pandas) for preprocessing and analysis [75, 76]. All experiments are configuration-driven (YAML) and each run is logged into a dedicated output directory to support traceability and reproducibility.¹

5.5 Index creation

All IVF-based indexes are trained offline using a fixed training set of size $N_{\text{train}} = 3,000,000$ passages sampled from the collection. All indexes share the same coarse quantizer configuration with `nlist=8192` inverted lists (clusters), so that comparisons isolate the effects of tiering and gating rather than differences in coarse partitioning. Unless stated otherwise, experiments use IVF-Flat and vary `nprobe` to control the effectiveness–efficiency trade-off (see Section 8).

5.5.1 IVF instead of HNSW: scalability and build-time considerations

In this work we adopt IVF-based indexes because the target collection (MS-MARCO passages) is large enough that *scalability and build-time* become first-order constraints. The Faiss guidelines note that graph-based indexes (e.g., HNSW) are a strong option when memory is not a constraint and the database is typically below $\sim 1\text{M}$ vectors, whereas beyond $\sim 10\text{M}$ vectors the *construction time* often

¹Code and experimental scripts are available at: github.com/comocomo39/TwoTierArchitecture.

becomes the main bottleneck. For larger collections, where some form of *compression* is needed to fit vectors in memory, IVF indexes are indicated as the practical choice because they naturally combine non-exhaustive search while preserving a simple and controllable efficiency knob through `nprobe` [58].

5.6 Search-time controls and experimental sweeps

To characterize the effectiveness–efficiency frontier, experiments sweep (i) the high-tier share α , (ii) ANN probing breadth (`nprobe`), and (iii) the low-tier activation settings that govern whether a fallback search is executed. The definition and implementation of the fallback/gating mechanism are discussed explicitly in RQ2. Architectural details of Virtual Partitioning, including the reuse of coarse IVF assignments across high- and low-tier passes, are also discussed in RQ2. This makes it possible to observe how ANN search effort and tiered architectures jointly shape the trade-off between ranking effectiveness and query-time cost under the same quality-induced partition. Two-Tier and Virtual Partitioning are evaluated under matched conditions (same split, same top- k , and comparable ANN parameters) to enable comparisons.

5.7 Measurements

Effectiveness is measured with ranking metrics such as NDCG@10 and MRR@10. Efficiency is quantified through query latency (average latency) and fallback dynamics (activation rate). All results are reported relative to the full-index baseline to highlight the net benefit of tiered retrieval. We evaluate all methods under the same topics/qrels and the same metric suite, using a single `pt.Experiment`-based harness so that differences reflect the retrieval architecture rather than the evaluation procedure. Runtime is measured by timing the retrieval call `transform` over the full query batch and reporting milliseconds per query.

Chapter 6

QualT5 quality signal: distribution, separability, and tiny vs base (RQ1)

RQ1. *How do QualT5-tiny and QualT5-base differ in their quality-score distributions, and how reliably do they induce a high/low partition of the corpus that is stable and interpretable?*

RQ1 evaluates whether QualT5 provides a reliable and interpretable signal for quality-aware corpus partitioning. The analysis is organized in three parts. First, we characterize the global distribution of quality scores (tiny vs. base) and discuss what the observed skewness implies for the meaning of “high quality” in this setting. Second, we study how the signal interacts with the coarse structure of the ANN index by analyzing variation across IVF inverted lists. Third, we quantify the downstream impact of using QualT5-tiny versus QualT5-base to induce the partition under matched retrieval conditions.

6.1 Global score distributions: skewness and limited separability at the high end

Figure 6.1 reports the global histograms of QualT5 scores for both model variants. The distributions are not approximately normal. Instead, they are strongly *left-skewed*: the bulk of passages receives scores concentrated near the upper end of the range, while a comparatively small fraction forms a long tail toward lower scores.

This shape has a direct interpretive consequence for tiering. When most of

the corpus is assigned scores near the upper bound, the quality estimator is most informative at identifying passages in the clearly low-quality tail, whereas it offers *reduced discriminative granularity* among passages already deemed “good enough”, particularly in the densely populated high-score region. In practice, percentile-based thresholding should therefore be interpreted primarily as isolating the worst $x\%$ of passages (those in the low-score tail), rather than as reliably ranking the entire corpus by fine-grained “degree of goodness” within the high-score region.

This perspective is consistent with the goal of this work. Tiered retrieval does not require that the high tier contains only exemplary passages; it requires that a relatively small subset of clearly low-quality content can be deprioritized without substantially harming effectiveness. However, the left-skewness also suggests a potential brittleness for very aggressive filtering: once thresholds cut into the high-score region where many passages are densely concentrated, the split separates items that are only weakly distinguishable under the quality signal, increasing the risk of misplacing useful passages into the low tier.

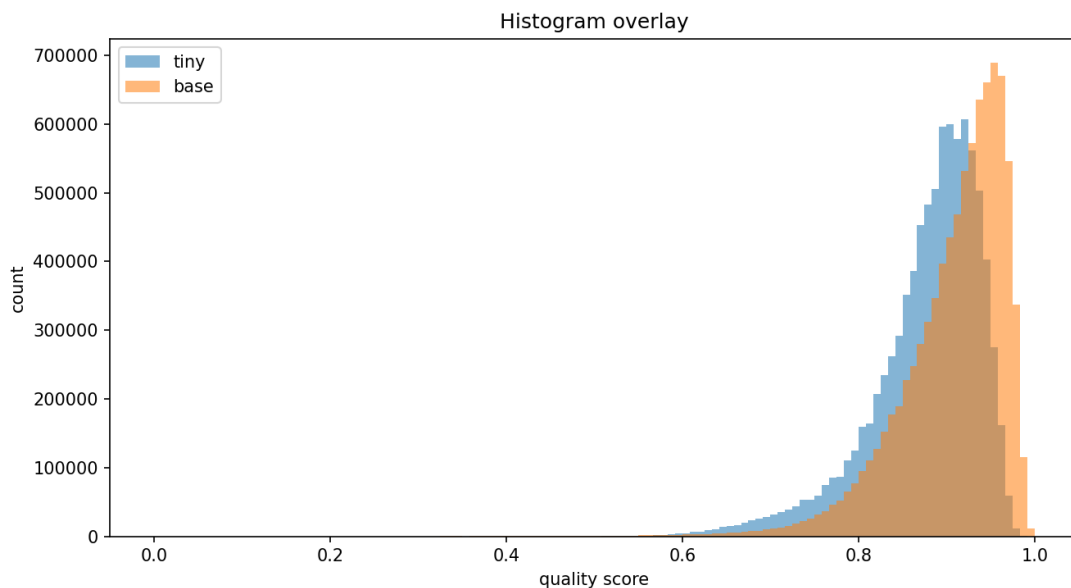


Figure 6.1: Global distribution of QualT5 scores for tiny and base. Both distributions are strongly left-skewed (mass near the upper bound with a tail toward low scores); base is more concentrated near the upper end.

6.1.1 Tiny vs. Base: calibration shift and concentration

The overlay reveals a consistent calibration shift between QualT5-tiny and QualT5-base. QualT5-base is more concentrated near the upper bound, with a sharper

peak and a distribution shifted to the right relative to QualT5-tiny. QualT5-tiny assigns slightly lower scores on average and exhibits broader dispersion.

This difference does not, by itself, imply that one model is superior. It does imply that the two variants are not directly comparable in absolute score values. For this reason, we define high/low tiers using percentile thresholds (matched partition sizes) rather than fixed score cutoffs, ensuring that tiny and base are compared under the same retained fraction in the high tier. At the same time, the stronger concentration of QualT5-base reinforces the earlier observation: in this corpus, the signal behaves primarily as a detector of the clearly low-quality tail, while offering limited discrimination among the many passages clustered near high scores.

6.2 Cluster-conditioned behavior: heterogeneity across IVF coarse partitions

Global distributions can obscure systematic differences across regions of the embedding space. To test whether the QualT5 signal behaves homogeneously with respect to the ANN coarse structure, we analyze it *within IVF inverted lists* of the *complete* index (built over the full corpus, without pruning or tiering). These lists represent the primary routing units at query time and therefore constitute the natural granularity at which quality-based partitioning may interact with ANN search.

Figure 6.2 summarizes this analysis by reporting, for each target low-tier share ($\text{low_share} \in \{0.20, 0.30, \dots, 0.80\}$), the distribution of *cluster-specific thresholds*. Each histogram bin counts how many IVF lists have a cluster-specific threshold t_c falling in that score range. Concretely, for each IVF list we compute the within-list quantile at the chosen low_share ; the histogram shows how these per-list thresholds vary across the index. The dashed vertical line denotes the *global threshold* that yields the same low-tier share when applied to the entire corpus.

Two patterns are evident. First, for any fixed low_share , cluster-specific thresholds exhibit substantial dispersion around the global cutoff, indicating pronounced cross-cluster heterogeneity: some lists contain passages that are systematically scored higher (requiring higher thresholds to assign the same low fraction), while others are systematically lower (requiring lower thresholds). Second, this dispersion is present for both QualT5 variants, despite their different global calibrations.

This heterogeneity matters for tiering in two ways. From a *partitioning* stand-

point, a single global threshold does not induce a uniform low-tier fraction within each IVF list; rather, it yields lists that are “low-tier heavy” and others that are “high-tier heavy.” From a *retrieval* standpoint, this creates an interaction between quality-based filtering and ANN routing: IVF search probes only a subset of lists per query, so the effective availability of high-tier candidates depends on whether the probed lists are high-tier heavy or low-tier heavy under the global split. Consequently, query-time variability can reflect not only query difficulty, but also how the query’s routed lists align with the list-level distribution of quality scores.

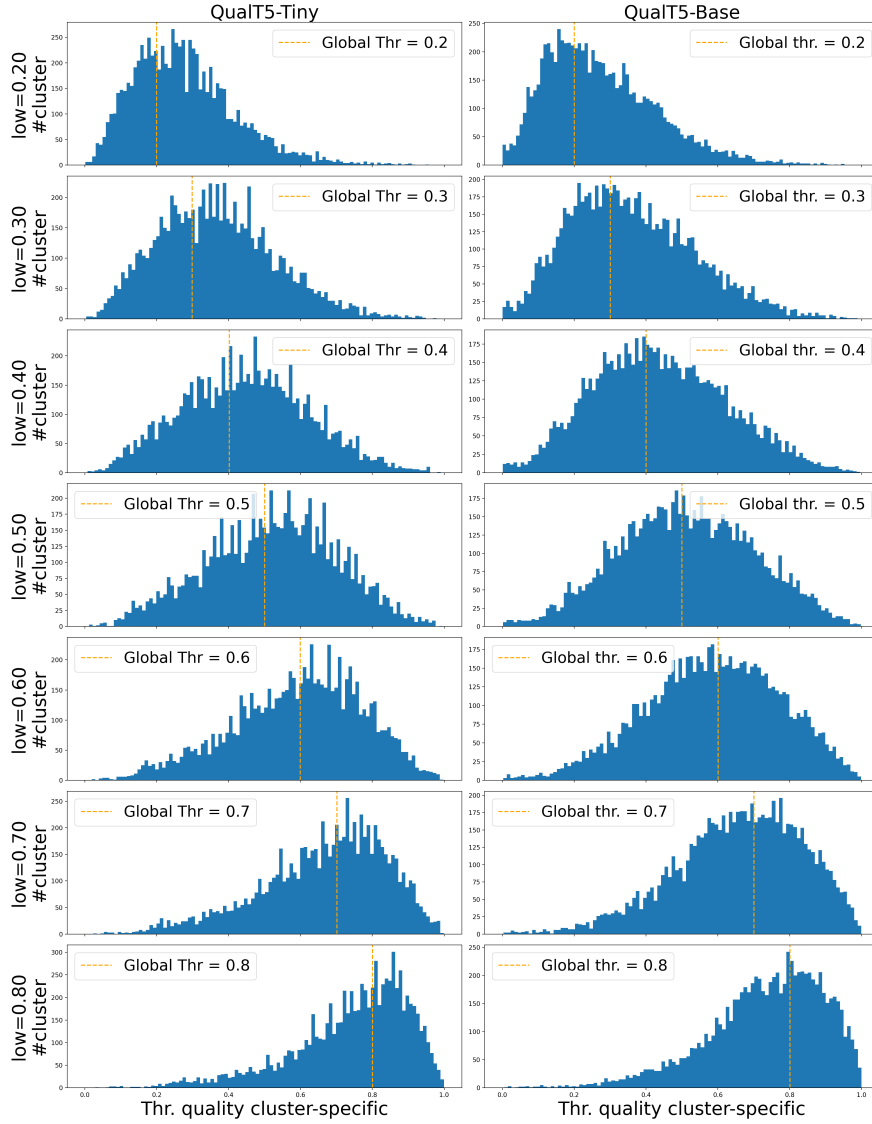


Figure 6.2: Distribution of cluster-specific thresholds required to obtain a fixed low-tier share within each IVF list of the complete index. The dashed line denotes the corresponding global threshold, highlighting cross-list heterogeneity.

6.3 Practical impact of the induced partitions (tiny vs. base)

The previous sections characterize QualT5 as a signal (global skewness and calibration) and as a system component that interacts with the IVF coarse structure (cluster-level heterogeneity). We now verify that the induced high/low splits translate into measurable differences in retrieval behavior under matched conditions, even in the simplest setting where retrieval is restricted to the high tier only (HighOnly VP). To this end, we report a controlled comparison where the retrieval architecture, ANN settings, encoder, and evaluation protocol are held constant, and only the quality estimator used to threshold the corpus changes (QualT5-tiny vs. QualT5-base).

Tables 6.1 and 6.2 report **HighOnly Virtual Partitioning** at `nprobe`=128 across multiple high-tier shares ($p \in \{20, 30, 40, 50, 60, 70, 80\}\%$), together with the complete-index baseline. Because the architecture and search effort are matched, differences in effectiveness and latency can be attributed to the quality signal’s ability to induce a useful partition.

Across most operating points, QualT5-base yields higher effectiveness than QualT5-tiny at comparable latency. This pattern is consistent with a more reliable separation between passages that can be safely deprioritized and passages that should remain readily accessible in the high tier. In line with the distributional analysis under left-skewness, the advantage of QualT5-base can be interpreted as improved robustness in identifying the low-quality tail, whereas QualT5-tiny appears to introduce more noise near the dense high-score region. In practice, such noise can move genuinely useful passages into the low tier, reducing the recall achievable when searching only the high tier at a given retained fraction. In addition, if quality assignments are uneven across IVF lists, the interaction with ANN routing (Section 6.2) can amplify variability in which high-tier candidates are reachable under constrained probing.

These results are not intended to replace the end-to-end effectiveness–efficiency frontier analysis in RQ3. Rather, they establish that the choice of quality estimator materially affects the induced partition and its downstream retrieval properties. This motivates using QualT5-base as the primary signal in subsequent experiments, while treating QualT5-tiny as a weaker baseline for quality-induced splits.

Method	Time (ms/q)	NDCG@10	MRR@10
HighOnly VP (20%)	3.2	0.1930	0.1773
HighOnly VP (30%)	4.7	0.2449	0.2222
HighOnly VP (40%)	5.8	0.2887	0.2600
HighOnly VP (50%)	7.4	0.3171	0.2834
HighOnly VP (60%)	8.8	0.3413	0.3038
HighOnly VP (70%)	9.9	0.3615	0.3202
HighOnly VP (80%)	10.8	0.3733	0.3291
Complete Index (Baseline)	11.2	0.3895	0.3415

Table 6.1: HighOnly Virtual Partitioning induced by QualT5-base (nprobe=128).

Method	Time (ms/q)	NDCG@10	MRR@10
HighOnly VP (20%)	3.2	0.1536	0.1422
HighOnly VP (30%)	4.7	0.1984	0.1815
HighOnly VP (40%)	5.8	0.2391	0.2168
HighOnly VP (50%)	7.4	0.2726	0.2440
HighOnly VP (60%)	8.8	0.3034	0.2700
HighOnly VP (70%)	9.9	0.3308	0.2935
HighOnly VP (80%)	10.8	0.3561	0.3149
Complete Index (Baseline)	11.2	0.3895	0.3415

Table 6.2: HighOnly Virtual Partitioning induced by QualT5-tiny (nprobe=128).

Chapter 7

Architectural design: Two-Tier vs Virtual Partitioning (RQ2)

RQ2. *What are the practical differences between a Two-Tier design (separate high/low indexes) and Virtual Partitioning (a single index with logical filtering), and how do these architectural choices affect latency, and fallback behavior under comparable retrieval settings?*

7.1 Rationale and comparison dimensions

Both architectures implement the same conceptual idea: queries are served from the high-quality tier whenever possible, while the low-quality tier is accessed only for a subset of queries deemed “uncertain” after the high-tier pass. RQ2 isolates *architectural* effects by comparing: (i) a **Two-Tier** pipeline with two physical FAISS indexes, and (ii) a **Virtual Partitioning** pipeline that maintains a single physical FAISS index and enforces the tier split via ID-based filtering. The comparison (discussed in RQ3) focuses on three dimensions: **(a)** query-time cost (average), **(b)** IR metrics (NDCG@10 and MRR@10), and **(c)** fallback dynamics (activation rate).

7.2 Two-Tier: two physical indexes and query-level fallback

In the Two-Tier design, two independent ANN indexes are constructed: one over the high-quality subset and one over the low-quality subset. At query time, the system first retrieves candidates from the high-tier index. A fallback decision is then applied *per query*; if fallback is triggered, the same query is executed on the low-tier index and the two candidate sets are merged into a single ranking.

Coarse assignment across tiers. In our Two-Tier implementation, the high-tier and low-tier indexes are built as two independent IVF structures and do not share the same coarse quantizer (centroid set). Therefore, the coarse assignment computed during the high-tier pass is not reused for the low-tier pass: if fallback is triggered, the low-tier search performs its own coarse assignment within the low-tier index.

Implementation. In the codebase, the Two-Tier pipeline is implemented by combining: (i) a FAISS-based retriever over the high-tier index and (ii) a FAISS-based retriever over the low-tier index, wrapped by a `TwoTier` transformer that performs gating, conditional low-tier execution, and result merging. The merge step sorts candidates by score and retains the final top- k results per query.

7.3 Virtual Partitioning: one physical index with logical filtering and shared coarse assignment

Virtual Partitioning maintains a single physical ANN index built over the full collection and enforces the high/low split at query time by restricting which document IDs are eligible for scoring. The system runs a high-tier pass first; if the gating policy triggers fallback, it executes a second pass over the complementary (low-tier) subset and merges the results.

Implementation. The virtual pipeline is implemented by `SharedMaskedTwoTier`, which combines (i) bitmap-based ID filtering through FAISS selectors and (ii) reuse of the IVF coarse assignment across high- and low-tier passes. The following paragraphs detail these two mechanisms.

Bitmap-based ID filtering (FAISS selectors). Tier membership is represented as boolean masks over the full corpus (`mask_high` and `mask_low`), aligned with FAISS internal vector IDs. For efficient membership checks, masks are

packed into bitmaps via `np.packbits` (little-endian), producing compact `uint8` arrays passed to `faiss.IDSelectorBitmap`. Each selector is bound to IVF search through a dedicated `faiss.SearchParametersIVF` object (`params_high` and `params_low`), sharing the same probing budget (`nprobe`). Because `IDSelectorBitmap` references the bitmap memory, the packed arrays are kept alive for the retriever lifetime to avoid invalidating the selector.

Shared coarse routing and filtered fine search via `search_preassigned`.

To reduce duplicated work across the two passes, Virtual Partitioning reuses the IVF coarse assignment. Given a batch of query vectors, the IVF quantizer is queried once to obtain the $P = \text{nprobe}$ closest centroids per query (`coarse_I`, `coarse_D`). The high-tier pass then calls `search_preassigned` with `params_high`, restricting scoring to the preselected inverted lists and to IDs admitted by the high-tier selector. If fallback is triggered for a subset of queries, the implementation slices the already computed preassignment to those queries and invokes `search_preassigned` again with `params_low`. Thus, fallback changes only the selector (high vs. low) while reusing the same routed lists, after which high- and low-tier candidates are merged by score and truncated to the final top- k .

7.4 Fallback (gating) mechanism: margin and entropy as uncertainty signals

Fallback is controlled by a query-level gating function that inspects the score distribution returned by the high-tier pass. Let $s_1 \geq s_2 \geq \dots$ denote the scores of the ranked candidates returned by the high-tier retrieval for a query. In our setup, FAISS returns Inner Product similarity scores, therefore higher values indicate better matches and the score-based uncertainty signals below are directly applicable. Two distinct uncertainty signals are used.

Margin (top-1 vs. top-2 gap). The margin signal considers the gap between the top-1 and top-2 scores, $\Delta = s_1 - s_2$. Fallback is triggered when $\Delta < m$, where m is a tunable threshold (**margin**). Intuitively, small margins indicate that the high-tier pass is not confidently separating the best candidate from its closest competitor, and therefore the low-tier pool is queried to increase coverage.

Entropy (dispersion over the top- k scores). The entropy signal measures how concentrated the high-tier scores are among the top candidates. Let $k =$

$\min(\text{topn_entropy}, \#\text{candidates})$. A softmax distribution is defined over the top- k scores:

$$p_i = \frac{\exp(s_i/\tau)}{\sum_{j=1}^k \exp(s_j/\tau)},$$

where τ is a temperature parameter (`tau`) controlling sharpness. The entropy is

$$H = -\sum_{i=1}^k p_i \log(p_i).$$

To make the threshold comparable across different k , the decision uses the maximum entropy $\log(k)$ as a reference: fallback is triggered when

$$H \geq \alpha \log(k),$$

where $\alpha \in [0, 1]$ is the `entropy_threshold` parameter. High entropy indicates that probability mass is spread across multiple candidates (i.e., the high-tier pass is uncertain), motivating a low-tier fallback.

7.5 Pros and cons of the two approaches (summary)

Two-Tier (physical partitioning). *Pros:* (i) searches a physically smaller high-tier index by default, which can reduce average latency when fallback is infrequent; (ii) allows tier-specific engineering and tuning (e.g., different index configurations) because the two tiers are independent artifacts. *Cons:* (i) when fallback triggers, the query effectively incurs a second ANN search; in our setup, this includes an additional coarse assignment because the two IVF indexes do not share a quantizer; (ii) increases operational overhead due to duplicated build, loading, and maintenance of two indexes.

Virtual Partitioning. *Pros:* (i) uses a single physical index, simplifying deployment and maintenance; (ii) supports flexible high-tier shares by updating ID masks (no index rebuild), and in IVF-based search can reuse the same coarse routing across high/low passes, reducing duplicated routing work under fallback. *Cons:* (i) efficiency gains depend on mask selectivity within the probed regions; if few high-tier items reside in the visited lists, the high-tier pass may be underpowered, increasing fallback activation or reducing effectiveness; (ii) sharing a single index constrains tier-specific optimization, since structural choices (index type,

compression, global parameters) are coupled across tiers.

Chapter 8

Final performance: effectiveness vs efficiency across partition thresholds (RQ3)

RQ3. *Across different high/low split thresholds, how do Two-Tier and Virtual Partitioning trade off effectiveness (e.g., NDCG@10, MRR@10) and efficiency (average latency), and how do they compare to a full-index baseline with no tiering?*

This chapter integrates (i) the architectural alternatives analyzed in RQ2 (Two-Tier with two physical indexes vs. Virtual Partitioning with a single index and logical filtering) and (ii) the partitioning regime characterized in RQ1 (quality-induced splits, using QualT5-base as the primary signal in subsequent analyses). The goal is to provide an end-to-end view of the effectiveness–efficiency frontier, highlighting where tiered retrieval preserves most of the baseline ranking quality while reducing average per-query cost.

8.1 Experimental setting and comparison protocol

All compared configurations share: (i) the same dataset and evaluation protocol, (ii) the same dense encoder (TAS-B) and *Inner Product* similarity (higher scores indicate better matches), and (iii) the same IVF-based ANN structure with search effort controlled by `nprobe`. Given a fixed split H_p/L_p (top- $p\%$ in the high tier), the analysis varies two main knobs:

- **High-tier size** ($p \in \{20, 30, 40, 50\}\%$ in the tables reported below).
- **ANN search effort** under two representative regimes: `nprobe`=128 (moderate budget) and `nprobe`=8192 (high budget).

For each regime, we compare three families of conditions: **(1) HighOnly** (search restricted to the high tier), **(2) tiering with fallback** (Two-Tier or Virtual Partitioning with margin- or entropy-based gating, using similar threshold grids to ensure a fair comparison), and **(3) baseline** (complete index with no tiering).

8.2 Margin vs. Entropy as Gating Signals

This section compares the two confidence signals used to gate the expensive retrieval step, namely *margin* and *entropy* (with formulation discussed in 7.4). In both cases, the goal is the same: reduce average latency by selectively activating the full search only when the query appears uncertain, while preserving ranking effectiveness.

Across all experiments, we report the trade-off between effectiveness (NDCG@10) and average query time. Points are annotated with the corresponding activation rate (percentage of queries that trigger the expensive step), and the *Full Index* baseline is shown as a reference operating point.

Threshold grid for the gating rules. For all comparisons, both gating functions were evaluated over a fixed grid of decision thresholds. For margin-based gating we sweep

$$\mathcal{T}_{\text{margin}} = \{0.1, 0.2, 0.5, 1.0, 1.5, 2.0, 2.5, 3.0, 3.5, 5.0\}, \quad (8.1)$$

while for entropy-based gating we sweep a uniform grid (with $\text{top-}k = 20$ and temperature $\tau = 1.0$)

$$\mathcal{T}_{\text{entropy}} = \{0.0, 0.1, 0.2, \dots, 0.9\}. \quad (8.2)$$

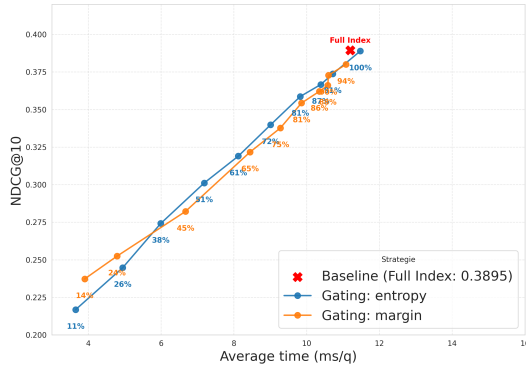
Each point on the curves corresponds to one threshold value in the respective grid, which implicitly determines the activation rate (i.e., the fraction of queries for which the expensive stage is triggered).

8.2.1 TwoTier: effectiveness–latency trade-offs under different splits

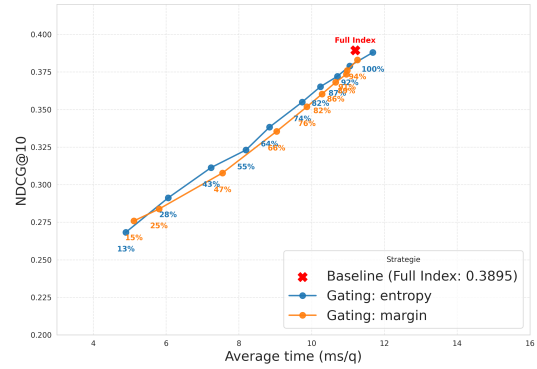
Figures 8.1 summarize the TwoTier results for multiple High/Low splits. Overall, the two gating signals produce *very similar behavior*. In several regions, the curves

are nearly overlapping, and when differences emerge, they are *local* (i.e., they depend on the split and on the operating point), with occasional crossings between the two lines.

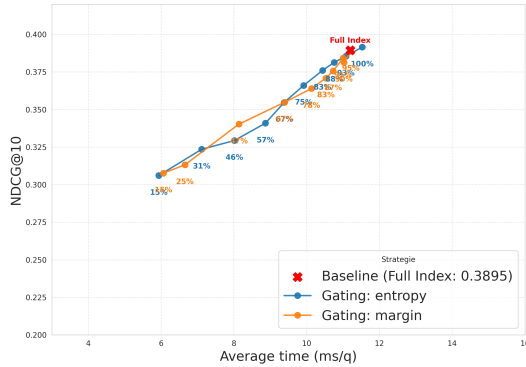
A consistent qualitative pattern is that the *activation rate* drives most of the trade-off: moving from low activation to high activation steadily increases NDCG@10 but also increases latency, and this trend holds for both signals. The choice between margin and entropy, instead, tends to affect only *second-order* variations. In practical terms, this indicates that both signals capture broadly comparable notions of query uncertainty in the TwoTier pipeline, and neither yields a systematically superior efficiency–quality profile across all splits.



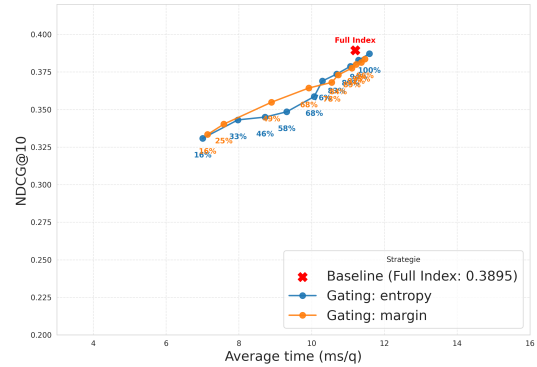
(a) Split 20/80 (TwoTier)



(b) Split 30/70 (TwoTier)



(c) Split 40/60 (TwoTier)



(d) Split 50/50 (TwoTier)

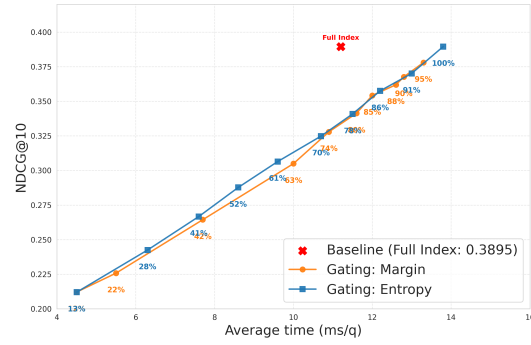
Figure 8.1: TwoTier trade-offs (NDCG@10 vs. average time) comparing margin- and entropy-based gating across different splits. Points are annotated with activation rates; the Full Index baseline is shown as reference.

8.2.2 Virtual Partitioning: robustness of the gating choice

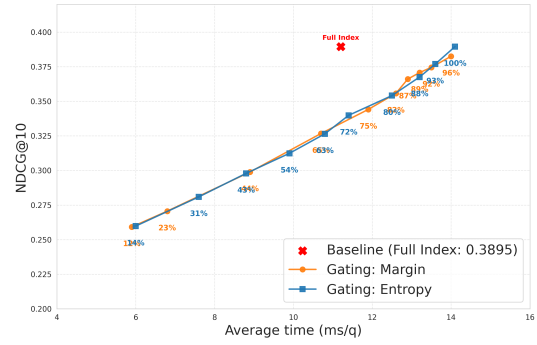
We observe the same high-level conclusion under Virtual Partitioning (Figures 8.2): margin and entropy again yield *comparable trade-offs*, with differences that are

small relative to the overall effect of changing the activation rate. In multiple splits, the two curves are nearly indistinguishable for most of the range, suggesting that the Virtual Partitioning mechanism is not particularly sensitive to which of the two confidence signals is used.

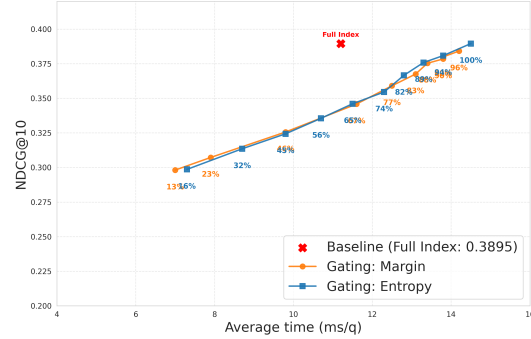
This is important from a design perspective: since neither signal consistently dominates, the gating choice can be guided by secondary criteria such as implementation simplicity, numerical stability, or interpretability. In the remainder of the experimental analysis, we therefore treat margin and entropy as *interchangeable gating options* and focus primarily on the operating point (activation budget) and on the structural choice between TwoTier and Virtual Partitioning.



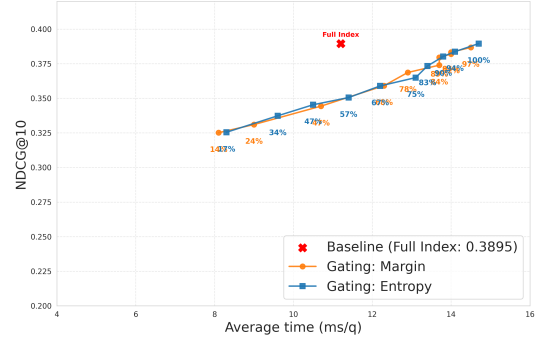
(a) Split 20/80 (Virtual Partitioning)



(b) Split 30/70 (Virtual Partitioning)



(c) Split 40/60 (Virtual Partitioning)



(d) Split 50/50 (Virtual Partitioning)

Figure 8.2: Virtual Partitioning trade-offs (NDCG@10 vs. average time) comparing margin- and entropy-based gating across different splits. As in TwoTier, the dominant effect is driven by activation rather than by the specific signal choice.

8.3 Moderate-cost regime: $nprobe=128$

8.3.1 Two-Tier: effectiveness–latency curve

Table 8.1 reports Two-Tier results for $nprobe=128$. The full-index baseline reaches $NDCG@10 = 0.3895$ and $MRR@10 = 0.3415$ at 11.2 ms/query. Restricting retrieval to the high tier only (**HighOnly**) yields a substantial latency reduction (e.g., 5.8 ms/query at $p = 50\%$) but introduces a clear effectiveness drop ($NDCG@10 = 0.3386$; $MRR@10 = 0.3009$). Enabling margin-based fallback recovers a meaningful portion of the lost quality: at $p = 50\%$ and $m = 1.0$, the system achieves $NDCG@10 = 0.3694$ and $MRR@10 = 0.3261$ at 10.0 ms/query, approaching the baseline while remaining cheaper on average.

Method	Time (ms/query)	NDCG@10	MRR@10
HighOnly (20%)	2.3	0.2222	0.2011
HighOnly (30%)	3.7	0.2761	0.2470
HighOnly (40%)	4.7	0.3115	0.2767
HighOnly (50%)	5.8	0.3386	0.3009
TwoTier (margin=0.5, 20%)	6.3	0.2896	0.2596
TwoTier (margin=0.5, 30%)	7.3	0.3209	0.2858
TwoTier (margin=0.5, 40%)	7.9	0.3421	0.3032
TwoTier (margin=0.5, 50%)	8.7	0.3591	0.3177
TwoTier (margin=1.0, 20%)	8.1	0.3229	0.2882
TwoTier (margin=1.0, 30%)	8.7	0.3426	0.3045
TwoTier (margin=1.0, 40%)	9.2	0.3561	0.3149
TwoTier (margin=1.0, 50%)	10.0	0.3694	0.3261
Complete Index (Baseline)	11.2	0.3895	0.3415

Table 8.1: Two-Tier with $nprobe=128$: effectiveness vs latency across p and margin-gating thresholds.

8.3.2 Virtual Partitioning: effectiveness–latency curve

Table 8.2 reports Virtual Partitioning under the same $nprobe=128$ regime. The qualitative behavior matches Two-Tier: HighOnly reduces latency but harms effectiveness, while fallback recovers quality. However, at matched (p, m) , Virtual Partitioning exhibits higher measured latency than Two-Tier (e.g., $p = 50\%$, $m = 1.0$: 12.3 ms/query for VP vs. 10.0 ms/query for Two-Tier), while still remaining below/around the baseline depending on the operating point. This suggests that,

in the $\text{nprobe}=128$ regime and for this implementation, the overhead of filtered search and the second pass can offset part of the expected advantage.

Method	Time (ms/q)	NDCG@10	MRR@10
HighOnly VP (20%)	3.2	0.1930	0.1773
HighOnly VP (30%)	4.7	0.2449	0.2222
HighOnly VP (40%)	5.8	0.2887	0.2600
HighOnly VP (50%)	7.4	0.3171	0.2834
Virtual Partitioning (margin=0.5, 20%)	7.8	0.2644	0.2389
Virtual Partitioning (margin=0.5, 30%)	8.8	0.2989	0.2689
Virtual Partitioning (margin=0.5, 40%)	10.1	0.3256	0.2909
Virtual Partitioning (margin=0.5, 50%)	10.6	0.3443	0.3066
Virtual Partitioning (margin=1.0, 20%)	9.7	0.3049	0.2740
Virtual Partitioning (margin=1.0, 30%)	10.7	0.3267	0.2919
Virtual Partitioning (margin=1.0, 40%)	11.4	0.3458	0.3071
Virtual Partitioning (margin=1.0, 50%)	12.3	0.3591	0.3180
Complete Index (Baseline)	11.2	0.3895	0.3415

Table 8.2: Virtual Partitioning with $\text{nprobe}=128$: effectiveness vs latency across p and margin-gating thresholds.

8.3.3 Practical reading of the frontier ($\text{nprobe}=128$)

Three operational observations emerge in the moderate-cost regime:

(i) HighOnly yields the largest speedups, but the effectiveness loss is substantial. For Two-Tier, moving from the baseline (11.2 ms/query, NDCG@10 0.3895, MRR@10 0.3415) to HighOnly at $p = 50\%$ (5.8 ms/query, NDCG@10 0.3386, MRR@10 0.3009) nearly halves latency while degrading both NDCG@10 and MRR@10.

(ii) Margin-gated fallback recovers a large fraction of the effectiveness gap with controlled overhead. At $p = 50\%$ and $m = 1.0$, Two-Tier improves from NDCG@10 0.3386 to 0.3694 and from MRR@10 0.3009 to 0.3261, while increasing latency from 5.8 to 10.0 ms/query, remaining below baseline cost.

(iii) In this regime, Two-Tier dominates Virtual Partitioning at multiple matched operating points. For the measured points, Two-Tier achieves lower

latency than Virtual Partitioning at the same (p, m) , while also achieving higher NDCG@10 and MRR@10 in corresponding settings (e.g., $p = 50\%$, $m = 0.5$ and $m = 1.0$).

8.4 High-cost regime: nprobe=8192

8.4.1 Two-Tier: approaching baseline quality under high search effort

With nprobe=8192 the baseline becomes dramatically more expensive (593.7 ms/query) but also more effective (NDCG@10 0.4104, MRR@10 0.3584). Under this regime, HighOnly provides large cost reductions but still exhibits a noticeable effectiveness gap (e.g., at $p = 50\%$: 313.6 ms/query, NDCG@10 0.3589, MRR@10 0.3172). Margin-gated fallback recovers additional effectiveness: at $p = 50\%$ and $m = 1.0$, Two-Tier reaches NDCG@10 0.3909 and MRR@10 0.3431 at 532.3 ms/query, i.e., close to baseline effectiveness at a lower cost.

Method	Time (ms/q)	NDCG@10	MRR@10
HighOnly (20%)	127.3	0.2375	0.2145
HighOnly (30%)	184.6	0.2946	0.2629
HighOnly (40%)	276.9	0.3294	0.2915
HighOnly (50%)	313.6	0.3589	0.3172
TwoTier (margin=0.5, 20%)	338.7	0.3069	0.2742
TwoTier (margin=0.5, 30%)	385.3	0.3405	0.3024
TwoTier (margin=0.5, 40%)	470.0	0.3598	0.3174
TwoTier (margin=0.5, 50%)	477.0	0.3794	0.3337
TwoTier (margin=1.0, 20%)	450.3	0.3424	0.3044
TwoTier (margin=1.0, 30%)	479.0	0.3632	0.3218
TwoTier (margin=1.0, 40%)	531.8	0.3758	0.3308
TwoTier (margin=1.0, 50%)	532.3	0.3909	0.3431
Complete Index (Baseline)	593.7	0.4104	0.3584

Table 8.3: Two-Tier with nprobe=8192: effectiveness vs latency across p and margin-gating thresholds.

8.4.2 Virtual Partitioning: comparison under high search effort

Table 8.4 reports Virtual Partitioning results at `nprobe`=8192. In this regime, Virtual Partitioning remains competitive relative to the baseline only in parts of the frontier. The measured points often show higher latency and/or smaller effectiveness recovery compared to Two-Tier (e.g., $p = 50\%$, $m = 1.0$: 665.2 ms/query and NDCG@10 0.3788, MRR@10 0.3337). At matched $p = 50\%$, Two-Tier with $m = 1.0$ attains higher NDCG@10 (0.3909) and higher MRR@10 (0.3431) at lower latency (532.3 ms/query).

Method	Time (ms/q)	NDCG@10	MRR@10
HighOnly VP (20%)	164.4	0.2063	0.1889
HighOnly VP (30%)	240.7	0.2605	0.2357
HighOnly VP (40%)	309.3	0.3061	0.2746
HighOnly VP (50%)	381.2	0.3368	0.2996
Virtual Partitioning (margin=0.5, 20%)	412.9	0.2809	0.2530
Virtual Partitioning (margin=0.5, 30%)	478.1	0.3165	0.2835
Virtual Partitioning (margin=0.5, 40%)	529.3	0.3448	0.3067
Virtual Partitioning (margin=0.5, 50%)	576.8	0.3641	0.3226
Virtual Partitioning (margin=1.0, 20%)	519.9	0.3215	0.2879
Virtual Partitioning (margin=1.0, 30%)	591.8	0.3459	0.3078
Virtual Partitioning (margin=1.0, 40%)	617.1	0.3656	0.3233
Virtual Partitioning (margin=1.0, 50%)	665.2	0.3788	0.3337
Complete Index (Baseline)	593.7	0.4104	0.3584

Table 8.4: Virtual Partitioning with `nprobe`=8192: effectiveness vs latency across p and margin-gating thresholds.

8.4.3 Practical reading of the frontier (`nprobe`=8192)

In the high-budget regime, tiering is no longer only a speed-up mechanism: it becomes a way to recover most of the baseline effectiveness while reducing a dominant query-time cost.

(i) HighOnly is very fast but too aggressive to preserve baseline effectiveness. At $p = 50\%$, HighOnly reduces cost substantially relative to the baseline but loses a non-trivial amount of both NDCG@10 and MRR@10.

(ii) **Two-Tier with fallback provides the closest-to-baseline operating points among the measured configurations.** In particular, $p = 50\%$ and $m = 1.0$ reaches NDCG@10 0.3909 vs. 0.4104 (baseline) and MRR@10 0.3431 vs. 0.3584 while reducing latency from 593.7 to 532.3 ms/query.

(iii) **Virtual Partitioning does not show a clear advantage over Two-Tier in these measurements.** At matched (p, m) , Virtual Partitioning tends to pay higher cost and achieve lower effectiveness recovery.

8.5 Comparative synthesis and recommended operating points

Across both **nprobe** regimes, the results suggest a stable pattern: (a) HighOnly is useful as an upper bound on speedup but incurs a substantial loss in NDCG@10 and MRR@10; (b) margin-gated fallback closes a large fraction of the effectiveness gap with moderate additional cost; (c) at the measured operating points, Two-Tier is more favorable than Virtual Partitioning in latency at matched (p, m) , and typically also achieves higher NDCG@10/MRR@10.

Without introducing external criteria, the reported measurements indicate the following practical points:

- **nprobe=128:** $p = 50\%$ with margin-gated fallback ($m = 1.0$) provides a near-baseline compromise at lower cost (Two-Tier: 10.0 ms/query, NDCG@10 0.3694, MRR@10 0.3261).
- **nprobe=8192:** $p = 50\%$ with margin-gated fallback ($m = 1.0$) is the closest-to-baseline point among those measured (Two-Tier: 532.3 ms/query, NDCG@10 0.3909, MRR@10 0.3431).

8.6 Frontier visualization: NDCG@10 and MRR@10 vs latency

For completeness, Figures 8.3–8.10 visualize the operating points as effectiveness–latency curves. NDCG@10 and MRR@10 are discussed jointly: in these experiments they move consistently, with MRR@10 typically mirroring the same trade-off trends observed in NDCG@10 (HighOnly drops, fallback recovery, baseline reference). This joint behavior indicates that tiering and fallback affect both top-ranked relevance (MRR@10) and graded top-10 gain (NDCG@10) in a coherent

way, rather than trading one metric for the other.

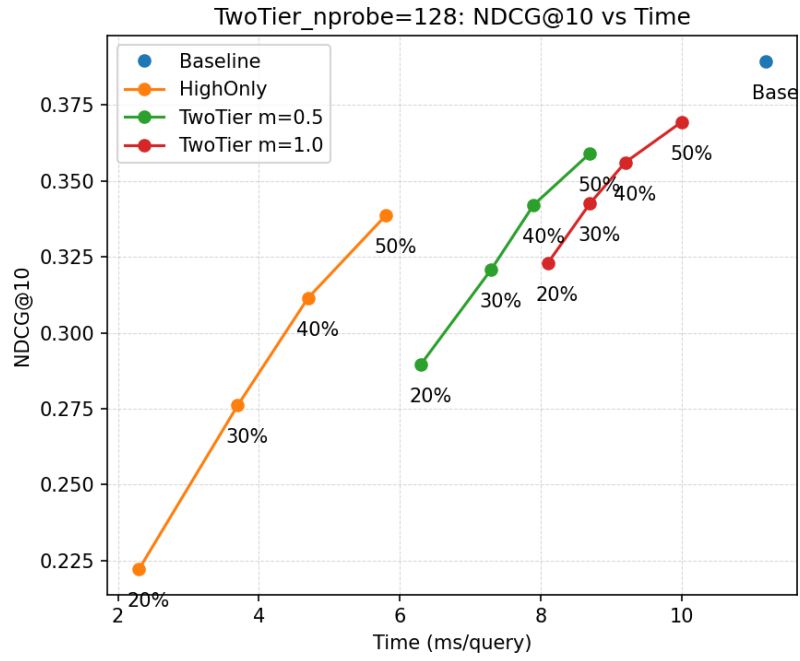


Figure 8.3: Two-Tier: NDCG@10 vs retrieval latency ($nprobe=128$).

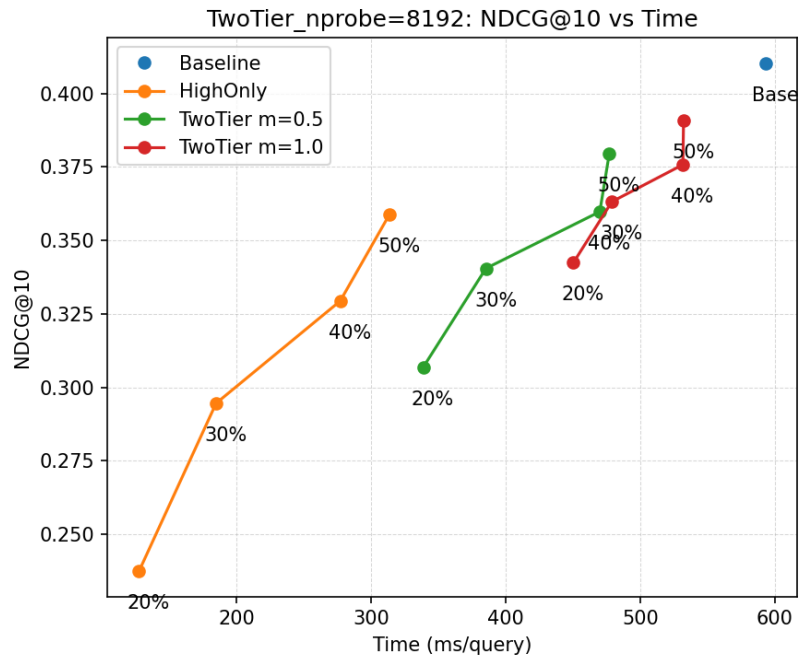


Figure 8.4: Two-Tier: NDCG@10 vs retrieval latency ($nprobe=8192$).

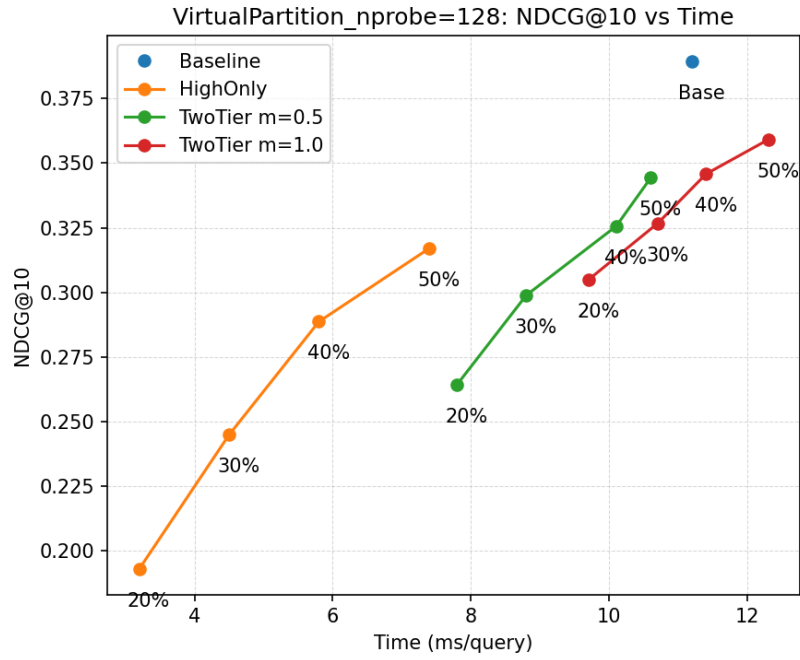


Figure 8.5: Virtual Partitioning: NDCG@10 vs retrieval latency (nprobe=128).

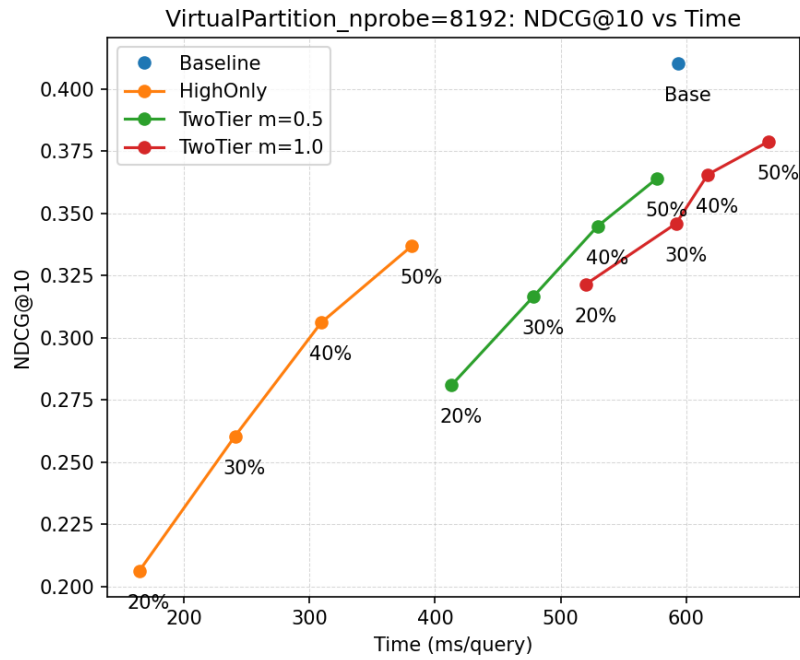


Figure 8.6: Virtual Partitioning: NDCG@10 vs retrieval latency (nprobe=8192).

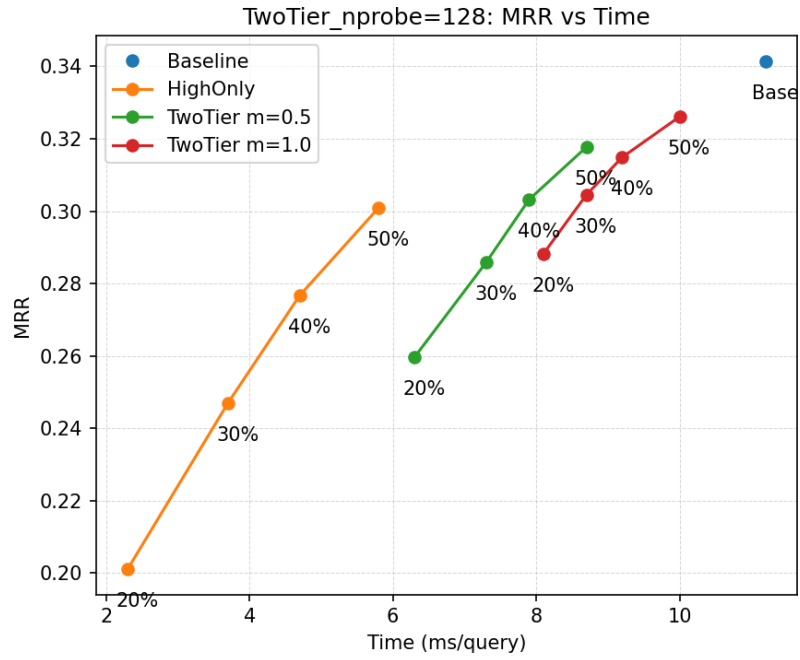


Figure 8.7: Two-Tier: MRR@10 vs retrieval latency ($nprobe=128$).

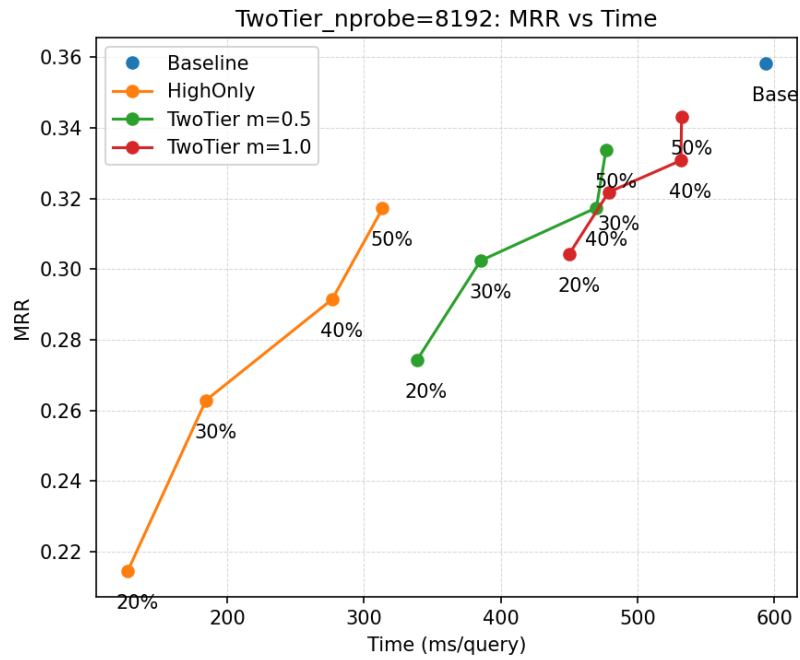


Figure 8.8: Two-Tier: MRR@10 vs retrieval latency ($nprobe=8192$).

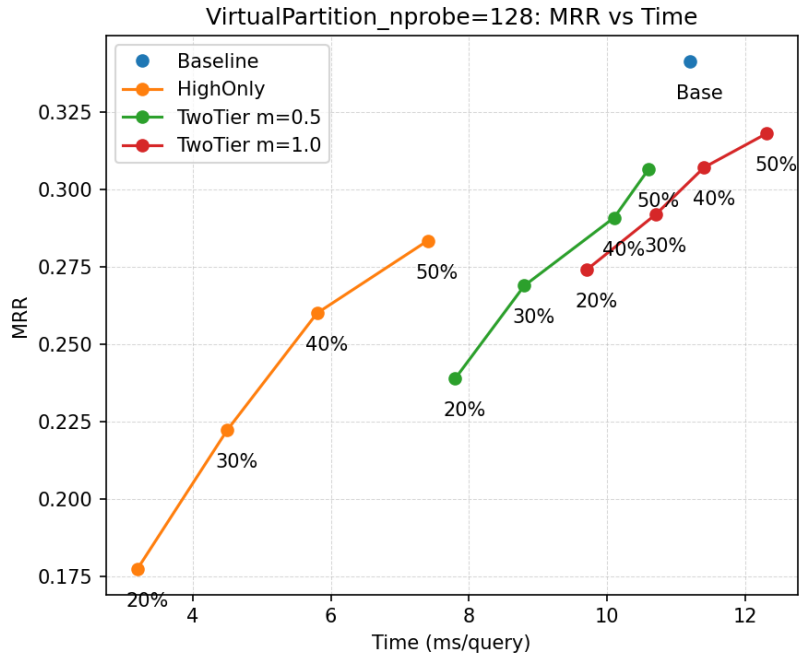


Figure 8.9: Virtual Partitioning: MRR@10 vs retrieval latency (nprobe=128).

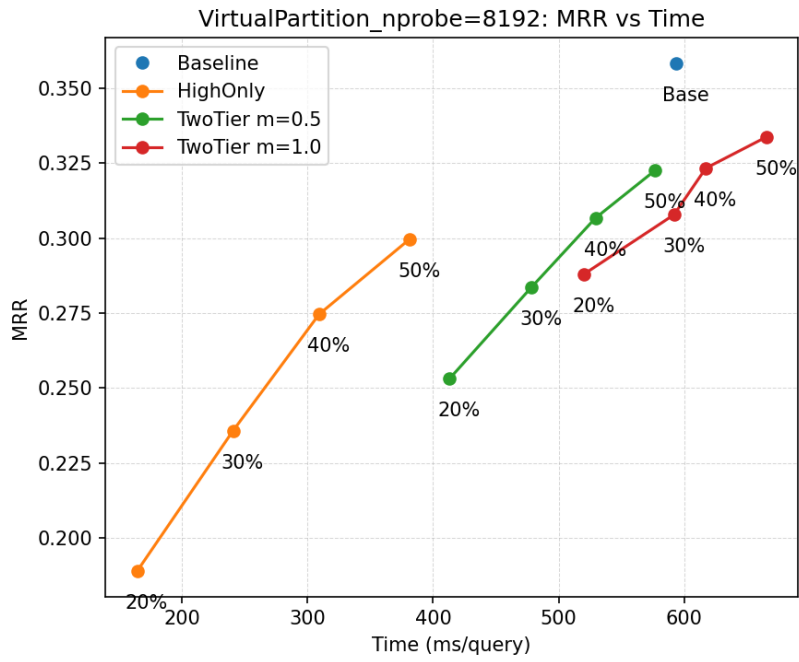


Figure 8.10: Virtual Partitioning: MRR@10 vs retrieval latency (nprobe=8192).

8.7 Answer to RQ3

Overall, quality-aware tiering enables operating points that reduce average latency relative to the full-index baseline, especially when retrieval is restricted to the high tier; however, HighOnly introduces a clear effectiveness drop that becomes substantial for aggressive splits. Margin-gated fallback recovers a large fraction of the lost effectiveness, moving operating points closer to the baseline with controlled additional cost. Across the measured configurations, Two-Tier yields a more favorable effectiveness–efficiency frontier than Virtual Partitioning for many (p, m) combinations, consistently for both `nprobe=128` and `nprobe=8192`, and this conclusion holds jointly for both NDCG@10 and MRR@10.

Chapter 9

Conclusions and Future works

This thesis explored the paradigm of **quality-aware tiered retrieval** as a solution to the operational tension between effectiveness and computational cost in modern dense retrieval systems. Starting from the observation that not all passages contribute equally to retrieval quality, we proposed and evaluated a methodology to partition the corpus into high- and low-quality tiers, employing selective fallback strategies to optimize the trade-off between latency and effectiveness.

9.1 Summary of findings

The research was guided by three fundamental questions, the main results of which are summarized below:

- **RQ1 – QualT5 Quality Signal:** The analysis of the signal produced by QualT5 showed a strongly **left-skewed** distribution. This indicates that the model is particularly effective at identifying and isolating low-quality content in the tail of the distribution, but offers limited resolution for discriminating between passages already deemed "good enough". The **QualT5-base** variant proved superior to QualT5-tiny, providing more reliable partitions and better separability between contents.
- **RQ2 – Architectural Design:** The comparison between the **Two-Tier** architecture (separate physical indexes) and **Virtual Partitioning** (single index with logical filtering) revealed that the Two-Tier design is generally more efficient in terms of latency and accuracy. Virtual Partitioning, while simplifying index management, suffers from significant overhead due to the handling of ID selectors and the requirement of a second scan pass, often resulting in slower performance than the full-index baseline under comparable search conditions.

- **RQ3 – Performance Frontiers:** End-to-end experiments demonstrated that tiered retrieval with selective fallback can recover a large fraction of the effectiveness lost with aggressive static pruning. In particular, the operating point with a **50% high-tier share** and a **margin-based fallback** threshold of **1.0** represented the best compromise, approaching the performance of the full-index baseline while reducing the average per-query latency.

9.2 General conclusions

In conclusion, this thesis demonstrates that quality-based corpus organization is a valid and robust alternative to both full indexing and traditional static pruning. The proposed approach allows for maintaining the high coverage necessary for difficult queries (via the low tier) while ensuring execution speed for the majority of the workload (via the high tier). From a systems perspective, implementation through separate physical indexes (*Two-Tier*) is the preferable choice to maximize latency savings.

9.3 Future works

Despite the promising results illustrated in this work, several research directions remain open to extend and refine the quality-aware tiered retrieval paradigm. A fundamental line of development concerns the generalization of the methodology beyond the MS MARCO benchmark, extending the experimental phase to other widely recognized passage datasets, such as TREC collections, to verify the system’s robustness across heterogeneous domains and search tasks. Furthermore, it would be of great interest to investigate the combination of *light pruning* techniques with the tiering strategies proposed here, exploring how a preliminary removal of the noisiest content could further optimize efficiency without compromising the coverage guaranteed by the low tier during fallback activations. From the perspective of text quality assessment, future research could focus on the creation of a specific *Quality score*, optimized for the analyzed corpus partitioning cases, while maintaining a strictly *query-independent* model nature to preserve the computational advantages of offline processes. In parallel, experimentation could be deepened by utilizing additional *gating* metrics and alternative confidence signals compared to margin and entropy, in order to identify even more precise decision criteria for the selective activation of the search in the low-quality tier. Finally, a crucial extension of the work lies in testing the effectiveness of

tiering on retrieval models other than TAS-B, with a particular focus on multi-representation architectures such as ColBERT, to analyze how the *late interaction* mechanism influences the selection dynamics between tiers and the overall effectiveness-efficiency trade-off.

Bibliography

- [1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. “Attention Is All You Need”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2017, pp. 5998–6008. URL: <https://arxiv.org/abs/1706.03762>.
- [2] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. *Introduction to Information Retrieval*. Cambridge, UK: Cambridge University Press, 2008. ISBN: 9780521865715.
- [3] Ricardo Baeza-Yates and Berthier Ribeiro-Neto. *Modern Information Retrieval: The Concepts and Technology behind Search*. 2nd ed. Boston, MA, USA: Addison-Wesley, 2011. ISBN: 9780321416919.
- [4] Ellen M. Voorhees and Donna K. Harman, eds. *TREC: Experiment and Evaluation in Information Retrieval*. Cambridge, MA, USA: MIT Press, 2005. ISBN: 9780262720487.
- [5] Kalervo Järvelin and Jaana Kekäläinen. “Cumulated Gain-Based Evaluation of IR Techniques”. In: *ACM Transactions on Information Systems* 20.4 (2002), pp. 422–446. DOI: 10.1145/582415.582418.
- [6] Robert S. Taylor. “The Process of Asking Questions”. In: *American Documentation* 13.4 (1962), pp. 391–396. DOI: 10.1002/asi.5090130405.
- [7] Nicholas J. Belkin, Robert N. Oddy, and Helen M. Brooks. “ASK for Information Retrieval: Part I. Background and Theory”. In: *Journal of Documentation* 38.2 (1982), pp. 61–71. DOI: 10.1108/eb026722.
- [8] Martin F. Porter. “An Algorithm for Suffix Stripping”. In: *Program* 14.3 (1980), pp. 130–137. DOI: 10.1108/eb046814.
- [9] Gerard Salton and Christopher Buckley. *Term-Weighting Approaches in Automatic Text Retrieval*. Republished/standard reference to classic term-weighting formulations. Information Processing & Management, 1988.

- [10] Stephen E. Robertson, Steve Walker, Susan Jones, Micheline Hancock-Beaulieu, and Mike Gatford. “Okapi at TREC-3”. In: *Proceedings of the Third Text REtrieval Conference (TREC-3)*. 1994.
- [11] Stephen Robertson and Hugo Zaragoza. “The Probabilistic Relevance Framework: BM25 and Beyond”. In: *Foundations and Trends in Information Retrieval*. Vol. 3. 4. 2009, pp. 333–389. DOI: 10.1561/15000000019.
- [12] Jay M. Ponte and W. Bruce Croft. “A Language Modeling Approach to Information Retrieval”. In: *Proceedings of SIGIR*. 1998, pp. 275–281. DOI: 10.1145/290941.291008.
- [13] Jimmy Lin and W. John Wilbur. “The Probabilistic Basis of Information Retrieval”. In: *Proceedings of the Human Language Technology Conference of the NAACL, Companion Volume: Short Papers*. 2007, pp. 11–14.
- [14] Po-Sen Huang, Xiaodong He, Jianfeng Gao, Li Deng, Alex Acero, and Larry Heck. “Learning Deep Structured Semantic Models for Web Search using Clickthrough Data”. In: *Proceedings of CIKM*. 2013, pp. 2333–2338. DOI: 10.1145/2505515.2505665.
- [15] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. “Dense Passage Retrieval for Open-Domain Question Answering”. In: *Proceedings of EMNLP*. 2020. DOI: 10.18653/v1/2020.emnlp-main.550.
- [16] Sepp Hochreiter and Jürgen Schmidhuber. “Long Short-Term Memory”. In: *Neural Computation* 9.8 (1997), pp. 1735–1780. DOI: 10.1162/neco.1997.9.8.1735.
- [17] Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. “Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation”. In: *arXiv preprint arXiv:1406.1078* (2014). URL: <https://arxiv.org/abs/1406.1078>.
- [18] Jonas Gehring, Michael Auli, David Grangier, Denis Yarats, and Yann N. Dauphin. “Convolutional Sequence to Sequence Learning”. In: *Proceedings of the 34th International Conference on Machine Learning (ICML)*. 2017, pp. 1243–1252. URL: <https://arxiv.org/abs/1705.03122>.

- [19] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. “Neural Machine Translation by Jointly Learning to Align and Translate”. In: *International Conference on Learning Representations (ICLR)*. 2015. URL: <https://arxiv.org/abs/1409.0473>.
- [20] Minh-Thang Luong, Hieu Pham, and Christopher D. Manning. “Effective Approaches to Attention-based Neural Machine Translation”. In: *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2015, pp. 1412–1421. DOI: 10.18653/v1/D15-1166.
- [21] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”. In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*. 2019, pp. 4171–4186. DOI: 10.18653/v1/N19-1423. URL: <https://arxiv.org/abs/1810.04805>.
- [22] Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. “Improving Language Understanding by Generative Pre-Training”. In: *OpenAI Technical Report* (2018). URL: https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf.
- [23] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. “Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer”. In: *Journal of Machine Learning Research* 21.140 (2020), pp. 1–67. URL: <https://jmlr.org/papers/v21/20-074.html>.
- [24] Bhaskar Mitra and Nick Craswell. “An Introduction to Neural Information Retrieval”. In: *Foundations and Trends in Information Retrieval* 13.1 (2018), pp. 1–126. DOI: 10.1561/15000000061. URL: <https://arxiv.org/abs/1709.07824>.
- [25] Jiafeng Guo, Yixing Fan, Qingyao Ai, and W. Bruce Croft. “A Deep Look into Neural Ranking Models for Information Retrieval”. In: *Information Processing & Management* 52.3 (2016), pp. 421–437. DOI: 10.1016/j.ipm.2015.04.003.
- [26] Bhaskar Mitra, Fernando Diaz, and Nick Craswell. “Learning to Match Using Local and Distributed Representations of Text for Web Search”. In: *Proceedings of the 26th International Conference on World Wide Web (WWW)*. 2017, pp. 1291–1299. DOI: 10.1145/3038912.3052579. URL: <https://arxiv.org/abs/1610.08136>.

- [27] Rodrigo Nogueira and Kyunghyun Cho. “Passage Re-ranking with BERT”. In: *Proceedings of the 2019 ACL Workshop on Deep Learning for Natural Language Processing*. 2019, pp. 1–7. DOI: 10.18653/v1/W19-6801. URL: <https://arxiv.org/abs/1901.04085>.
- [28] Nick Craswell, Bhaskar Mitra, Jimmy Lin, Avishek Anand, and Ian Soboroff. *Information Retrieval: Implementing and Evaluating Search Engines*. Foundations and Trends in Information Retrieval. Now Publishers, 2021. URL: <https://ciir.cs.umass.edu/irbook/>.
- [29] Zellig S. Harris. “Distributional Structure”. In: *Word* 10.2–3 (1954), pp. 146–162. DOI: 10.1080/00437956.1954.11659520.
- [30] J. R. Firth. “A Synopsis of Linguistic Theory 1930–1955”. In: *Studies in Linguistic Analysis*. Blackwell, 1957.
- [31] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. “Efficient Estimation of Word Representations in Vector Space”. In: *Proceedings of Workshop at ICLR*. 2013. URL: <https://arxiv.org/abs/1301.3781>.
- [32] Jeffrey Pennington, Richard Socher, and Christopher D. Manning. “GloVe: Global Vectors for Word Representation”. In: *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2014. DOI: 10.3115/v1/D14-1162. URL: <https://aclanthology.org/D14-1162/>.
- [33] Gilbert Strang. *Introduction to Linear Algebra*. 5th ed. Wellesley-Cambridge Press, 2016.
- [34] Gerard Salton, Anita Wong, and Chung-Shu Yang. “A Vector Space Model for Automatic Indexing”. In: *Communications of the ACM* 18.11 (1975), pp. 613–620. DOI: 10.1145/361219.361220.
- [35] Gerard Salton and Michael J. McGill. *Introduction to Modern Information Retrieval*. McGraw-Hill, 1983.
- [36] Sanjeev Arora, Yingyu Liang, and Tengyu Ma. “A Simple but Tough-to-Beat Baseline for Sentence Embeddings”. In: *International Conference on Learning Representations (ICLR)* (2017). URL: <https://arxiv.org/abs/1602.06023>.
- [37] Nicola Tonellotto. “Lecture Notes on Neural Information Retrieval”. In: *arXiv preprint arXiv:2207.13443* (2022). DOI: 10.48550/arXiv.2207.13443. URL: <https://arxiv.org/abs/2207.13443>.

- [38] Jiafeng Guo, Yixing Fan, Qingyao Ai, and W. Bruce Croft. “A Deep Relevance Matching Model for Ad-hoc Retrieval”. In: *Proceedings of the 25th ACM International on Conference on Information and Knowledge Management (CIKM '16)*. ACM, 2016, pp. 55–64. DOI: 10.1145/2983323.2983769.
- [39] Chenyan Xiong, Zhuyun Dai, Jamie Callan, Zhiyuan Liu, and Russell Power. “End-to-End Neural Ad-hoc Ranking with Kernel Pooling”. In: *Proceedings of the 40th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '17)*. ACM, 2017. DOI: 10.1145/3077136.3080809.
- [40] Kai Hui, Andrew Yates, Klaus Berberich, and Gerard de Melo. “PACRR: A Position-Aware Neural IR Model for Relevance Matching”. In: *arXiv preprint arXiv:1704.03940* (2017). To appear in EMNLP 2017 (as stated by the authors on arXiv). DOI: 10.48550/arXiv.1704.03940. URL: <https://arxiv.org/abs/1704.03940>.
- [41] Omar Khattab and Matei Zaharia. “ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT”. In: *arXiv preprint arXiv:2004.12832* (2020). DOI: 10.48550/arXiv.2004.12832. URL: <https://arxiv.org/abs/2004.12832>.
- [42] Lee Xiong, Chenyan Xiong, Ye Li, Kwok-Fung Tang, Jialin Liu, Paul Bennett, Junaid Ahmed, and Arnold Overwijk. “Approximate Nearest Neighbor Negative Contrastive Learning for Dense Text Retrieval”. In: *arXiv preprint arXiv:2007.00808* (2020). arXiv: 2007.00808 [cs.IR]. URL: <https://arxiv.org/abs/2007.00808>.
- [43] Jingtao Zhan, Jiaxin Mao, Yiqun Liu, Min Zhang, and Shaoping Ma. “Optimizing Dense Retrieval Model Training with Hard Negatives”. In: *arXiv preprint arXiv:2104.08051* (2021). arXiv: 2104.08051 [cs.IR]. URL: <https://arxiv.org/abs/2104.08051>.
- [44] Yingqi Qu, Yuchen Ding, Jing Liu, Kai Liu, Ruiyang Ren, Wayne Xin Zhao, Daxiang Dong, Hua Wu, and Haifeng Wang. “RocketQA: An Optimized Training Approach to Dense Passage Retrieval for Open-Domain Question Answering”. In: *arXiv preprint arXiv:2010.08191* (2020). NAACL 2021. arXiv: 2010.08191 [cs.CL]. URL: <https://arxiv.org/abs/2010.08191>.
- [45] Sebastian Hofstätter, Sheng-Chieh Lin, and Allan Hanbury. “Efficiently Teaching an Effective Dense Retriever with Balanced Topic-Aware Sampling”. In: *Proceedings of the 44th International ACM SIGIR Conference on Research*

- and Development in Information Retrieval (SIGIR)*. 2021. DOI: 10.1145/3404835.3462891.
- [46] Samuel Humeau, Kurt Shuster, Marie-Anne Lachaux, and Jason Weston. “Poly-encoders: Architectures and Pre-training Strategies for Fast and Accurate Multi-sentence Scoring”. In: *arXiv preprint arXiv:1905.01969* (2019). URL: <https://arxiv.org/abs/1905.01969>.
 - [47] Luyu Gao, Zhuyun Dai, and Jamie Callan. “COIL: Revisit Exact Lexical Match in Information Retrieval with Contextualized Inverted List”. In: *arXiv preprint arXiv:2104.07186* (2021). NAACL 2021. DOI: 10.48550/arXiv.2104.07186. URL: <https://arxiv.org/abs/2104.07186>.
 - [48] C. J. van Rijsbergen. *Information Retrieval*. 2nd ed. Butterworths, 1979.
 - [49] Ellen M. Voorhees. “The TREC-8 Question Answering Track”. In: *Proceedings of the Second International Conference on Language Resources and Evaluation (LREC 2000)*. 2000.
 - [50] Olivier Chapelle, Donald Metzler, Ya Zhang, and Pierre Grinspan. “Expected Reciprocal Rank for Graded Relevance”. In: *Proceedings of the 18th ACM Conference on Information and Knowledge Management (CIKM '09)*. ACM, 2009, pp. 621–630. DOI: 10.1145/1645953.1646033.
 - [51] Alistair Moffat and Justin Zobel. “Rank-biased precision for measurement of retrieval effectiveness”. In: *ACM Transactions on Information Systems* 27.1 (2008), pp. 1–27.
 - [52] Nick Craswell, Bhaskar Mitra, Emine Yilmaz, and Daniel Campos. “Overview of the TREC 2020 Deep Learning Track”. In: *Proceedings of the Twenty-Ninth Text REtrieval Conference (TREC 2020)*. Vol. 1266. NIST Special Publication. National Institute of Standards and Technology (NIST), 2020. DOI: 10.6028/NIST.SP.1266.deep-overview.
 - [53] Jeff Johnson, Matthijs Douze, and Hervé Jégou. *Billion-scale similarity search with GPUs*. 2017. arXiv: 1702.08734 [cs.LG].
 - [54] Kuo Chang, Shubham Mishra, Craig Macdonald, and Sean MacAvaney. “Neural Passage Quality Estimation for Static Pruning”. In: *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*. 2024. DOI: 10.1145/3626772.3657765. URL: <https://arxiv.org/abs/2407.12170>.

- [55] David Carmel, Daniel Cohen, Ronald Fagin, Eli Farchi, Michael Herscovici, Yoelle S. Maarek, and Aya Soffer. “Experiments with Index Pruning”. In: *Proceedings of the Tenth Text REtrieval Conference (TREC 2001)*. 2001. URL: <https://trec.nist.gov/pubs/trec10/papers/ibm.prune.pdf>.
- [56] Tri Nguyen, Mir Rosenberg, Xia Song, Jianfeng Gao, Saurabh Tiwary, Rangan Majumder, and Li Deng. “MS MARCO: A Human Generated Machine Reading COMprehension Dataset”. In: *arXiv preprint arXiv:1611.09268* (2016). URL: <https://arxiv.org/abs/1611.09268>.
- [57] Diederik P. Kingma and Jimmy Ba. “Adam: A Method for Stochastic Optimization”. In: *arXiv preprint arXiv:1412.6980* (2015). URL: <https://arxiv.org/abs/1412.6980>.
- [58] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Greg Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, and Hervé Jégou. “The Faiss library”. In: *arXiv preprint arXiv:2401.08281* (2024).
- [59] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. “Product Quantization for Nearest Neighbor Search”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33.1 (2011), pp. 117–128. DOI: 10.1109/TPAMI.2010.57.
- [60] Artem Chirkin. *Accelerated Vector Search: Approximating with NVIDIA cuVS Inverted Index (IVF-Flat)*. NVIDIA Technical Blog. Accessed: 2026-02-09. Oct. 2023. URL: <https://developer.nvidia.com/blog/accelerated-vector-search-approximating-with-nvidia-cuvs-ivf-flat/>.
- [61] Yu. A. Malkov and D. A. Yashunin. “Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs”. In: *arXiv preprint arXiv:1603.09320* (2018). Version 4 (last revised Aug 2018). DOI: 10.48550/arXiv.1603.09320. URL: <https://arxiv.org/abs/1603.09320>.
- [62] Ryan McDermott. *What’s The Story With HNSW?* Medium (Towards Data Science). Accessed: 2026-02-09. Feb. 2024. URL: <https://medium.com/data-science/whats-the-story-with-hnsw-d1402c37a44e>.
- [63] facebookresearch. *Faiss Wiki: Speed vs. accuracy (index parameters and trade-offs)*. GitHub Wiki. Accessed 2026-01-16. URL: <https://github.com/facebookresearch/faiss/wiki/Indexing-1M-vectors>.

- [64] Tianqi Ge, Kaiming He, Qifa Ke, and Jian Sun. “Optimized Product Quantization”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 36.4 (2014), pp. 744–755. DOI: 10.1109/TPAMI.2013.240.
- [65] *Similarity Search with IVFPQ*. Towards Data Science (Medium). Accessed: 2026-02-09. URL: <https://towardsdatascience.com/similarity-search-with-ivfpq-9c6348fd4db3/>.
- [66] *ir_datasets: MSMARCO (passage)* — *msmarco-passage/dev/small*. URL: <https://ir-datasets.com/msmarco-passage.html#msmarco-passage/dev/small> (visited on 01/17/2026).
- [67] pyterrier-quality. *qt5-tiny*. Hugging Face model repository. URL: <https://huggingface.co/pyterrier-quality/qt5-tiny>.
- [68] pyterrier-quality. *qt5-base*. Hugging Face model repository. URL: <https://huggingface.co/pyterrier-quality/qt5-base>.
- [69] pyterrier-quality. *qt5-base.msmarco-passage.cache*. Hugging Face dataset (pre-computed cache). URL: <https://huggingface.co/datasets/pyterrier-quality/qt5-base.msmarco-passage.cache>.
- [70] pyterrier-quality. *qt5-tiny.msmarco-passage.cache*. Hugging Face dataset (pre-computed cache). URL: <https://huggingface.co/datasets/pyterrier-quality/qt5-tiny.msmarco-passage.cache>.
- [71] Terrier Team. *pyterrier-quality: Quality Estimation for PyTerrier*. GitHub repository. URL: <https://github.com/terrierteam/pyterrier-quality>.
- [72] Sebastian Hofstätter. *distilbert-dot-tas_b-b256-msmarco*. Hugging Face model repository. URL: https://huggingface.co/sebastian-hofstaetter/distilbert-dot-tas_b-b256-msmarco.
- [73] FAISS contributors. *FAISS: A Library for Efficient Similarity Search and Clustering of Dense Vectors*. GitHub repository. URL: <https://github.com/facebookresearch/faiss>.
- [74] Terrier Team. *pyterrier_dr: Dense Retrieval Extensions for PyTerrier*. GitHub repository. URL: https://github.com/terrierteam/pyterrier_dr.
- [75] NumPy Developers. *NumPy: The Fundamental Package for Scientific Computing with Python*. GitHub repository. URL: <https://github.com/numpy/numpy>.
- [76] pandas development team. *pandas: Powerful Data Structures for Data Analysis, Time Series, and Statistics*. GitHub repository. URL: <https://github.com/pandas-dev/pandas>.

